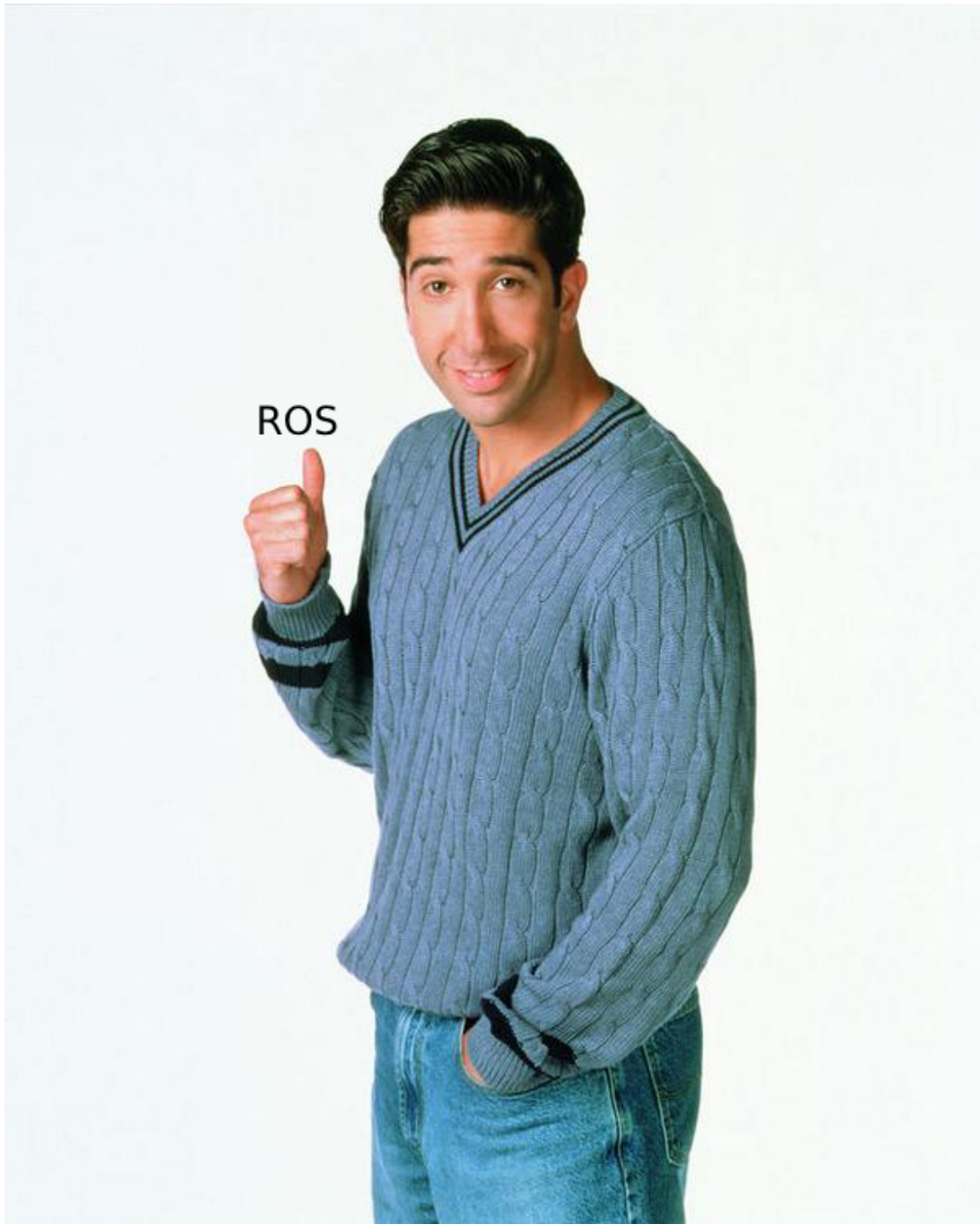


ROS2 WORKSHOP



Contents

1	Module 1: Workshop Rundown	2
1.1	Welcoming message	2
1.2	Content Overview	2
1.3	VM installation for linux and ROS	5
2	Module 2: Introduction to ROS	21
2.1	ROS 2 Humble Hawksbill	22
2.2	Introductory exercises for ROS2	22
2.3	Baby steps	22
3	Module 3: File structure	24
3.1	Creating ROS2 workspace	25
3.2	ROS Packages (Python)	26
3.3	Making your own ROS package	27
3.4	Python Packages: setup.py and setup.cfg	28
3.5	Package XML	29
4	Module 4: Computational Graph	31
4.1	Nodes, Subscriber, Publisher, Topics and Messages	32
4.2	Actions and Services	34
4.3	Basic Tutorials for all concepts	35
5	Module 5: Basic Exercises	36
5.1	Controlling Turtlebot in Gazebo	36
5.2	Publishers and Subscribers: PIG LATIN	38
5.3	Fire and floods disaster land robot State Machine	41
6	Module 6: Intermediate Exercises	45
6.1	Maze Mapping	45
6.2	Perception	48
7	Common Issues	53
7.1	SelectableGroups dict interface is deprecated	53
7.2	turtlebot_install.sh cannot find catkin	53
7.3	AMENT_PREFIX_PATH path does not exist	53

1 Module 1: Workshop Rundown

1.1 Welcoming message

HELLO MY PRETTIES

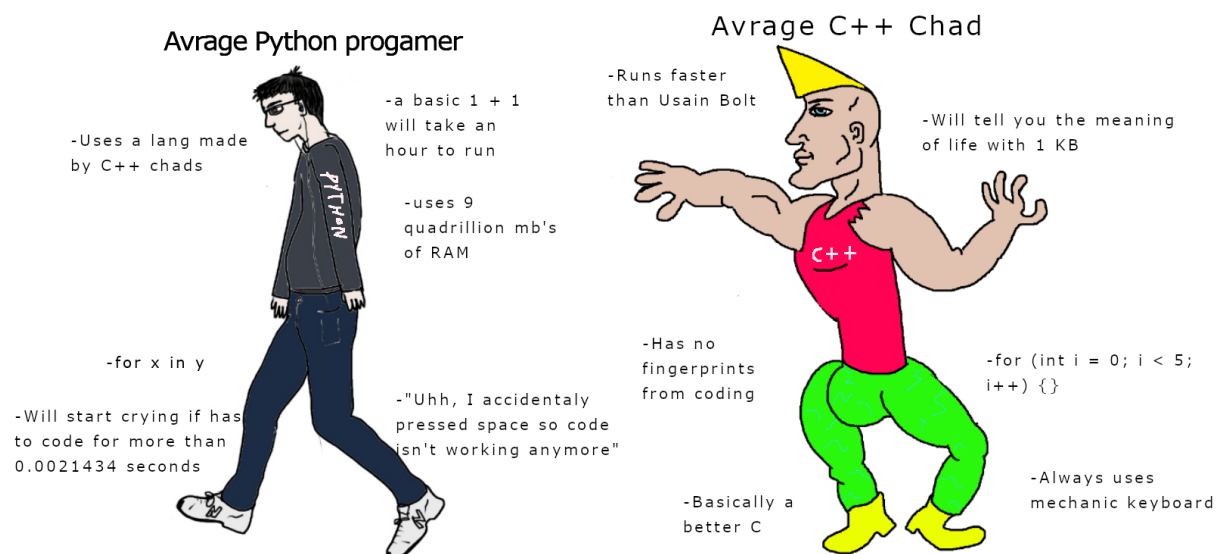
The University of Sydney Robotics Club welcomes all participants to the ROS workshop! We hope that you have managed to set up your VM on your personal laptops. **Now onto ROSSSSSSSSSSSS**



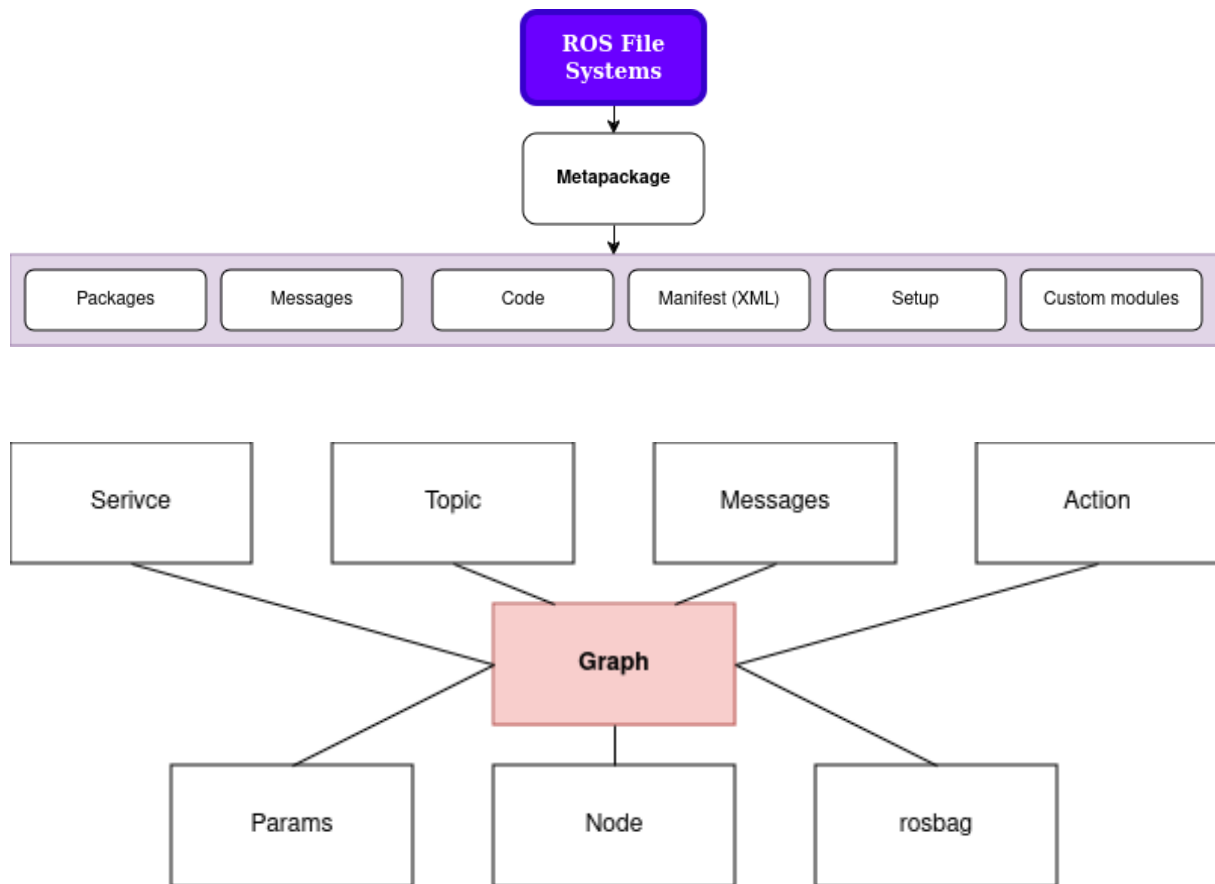
1.2 Content Overview

In this subsection, we will give you a rundown of how you should progress through this workshop. We don't expect everyone to finish everything, but we do expect that you put in effort so that, by the end, you have learnt enough of the basics of ROS to start your own robotics projects.

For 2nd year Mechatronics students, this workshop is extremely beneficial when you take MTRX3760 in your third year as a lot of modules you are doing now may relate to that course. However, the major difference is that for this workshop, you are working with **Python** not **C++** so you are not obliged to learn Object Oriented Programming (**hint hint**).



You will first be introduced to two main levels of **ROS architecture: file structure and computational graphs**.



The exercises we give you will allow you to have a firm grasp of the basics with progressive difficulty and then actual problems are given for you to solve with the basics you have learned.

- **Introductory and Basics:** *Getting you to learn the basic building blocks of ROS*

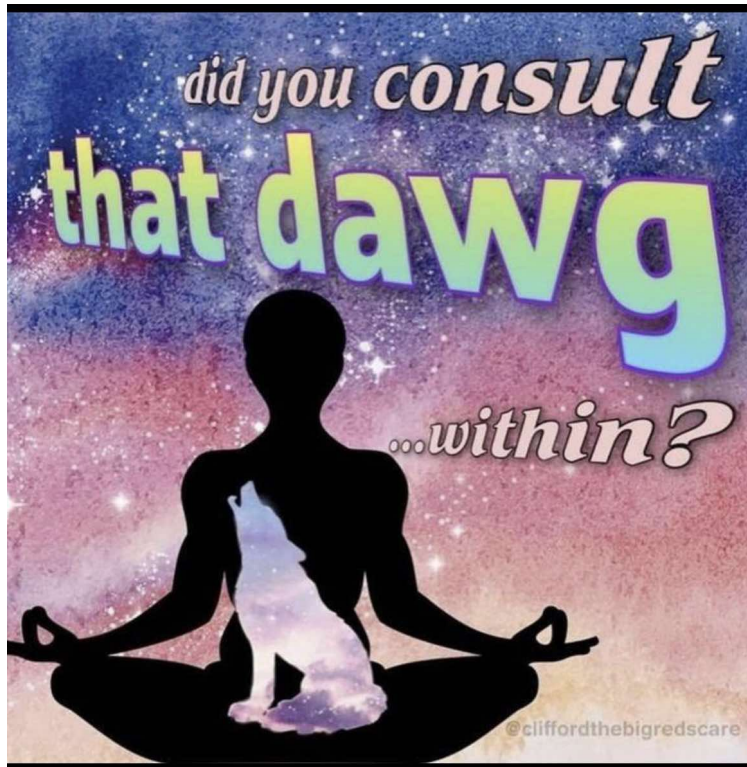
- Example codes are given to you with detailed comments. You are tasked to run these codes and understand how they work by using different ROS2 commands to explore. We basically giving you some basic building blocks to mess around.
- You are then tasked to write some simple codes similar to the ones we give you and solve some other tasks. These should not take you too long.

- **Intermediate:**

- This is where you will get familiarised with some ROS and robotics application fields and such as sensors, driving, navigation, maybe perception with a little dash of Computer Vision.
- We will give you some template codes for these tasks and it is your job to be able to produce the desired outputs. Even if you did not produce the results we want, trying to speculate and understand where you went wrong will give you some debugging experience that will benefit you in the long term as Engineers.
- This section can be quite hefty for some of you but it is critical for you to as when you progress onto the last part of the workshop . Utilising the application areas in this section, you will see how your modules translate into simulation in the simulation. Even with rigorous testing on your software, translating into real world may not be as smooth as you think and we want you to experience that in the next section of the workshop.

- **Challenge: Turtlebot Gazebo**

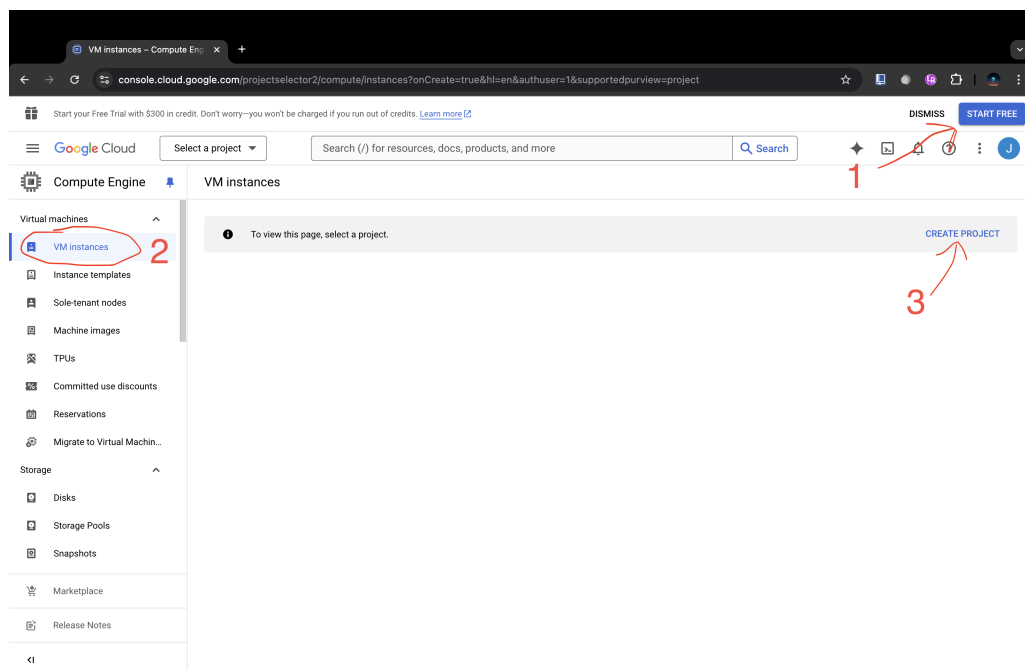
- We will hand you a comprehensive set of small objectives that you must be able to complete with Turtlebot in Gazebo, integrating what you have learnt so far about structuring ROS systems and using OpenCV.
- The challenge is done in simulation since we do not enough hardware for you (*Shivam was lazy to set it up, dont blame me*).
- The challenge awaits, are you ready to bring out that DAWG in you ???



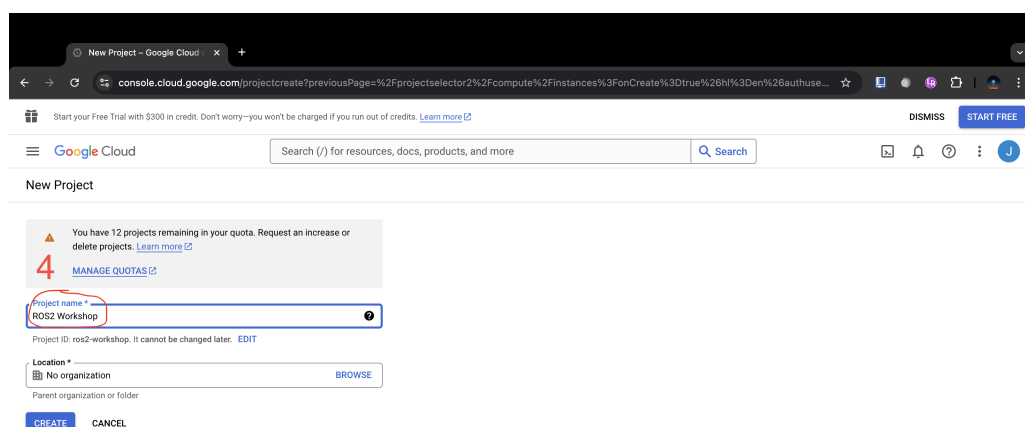
1.3 VM installation for linux and ROS

Let us first set up the environment you will be working in... using the power of Google Cloud's Compute Engine!

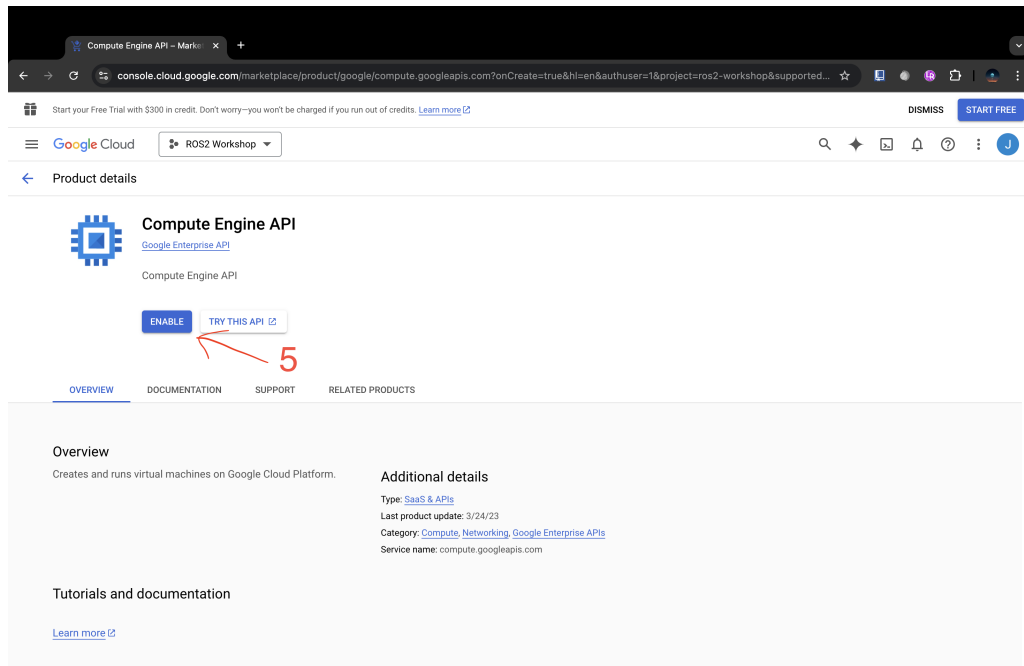
1. Go to <https://console.cloud.google.com/compute>. To begin, you will need to start a free 90-day trial with Google Cloud where you get 300USD credits for free! Create your account and start your free trial before proceeding. You will need an email and payment information for this. Once the trial has ended, if your VMs are no longer active, they won't charge you a single cent.
2. Go to your VM Instances.
3. Create a new project. This is how your cloud services are organised together. You will be the owner of this project, and can invite others to work on it and assign roles with specific permissions.



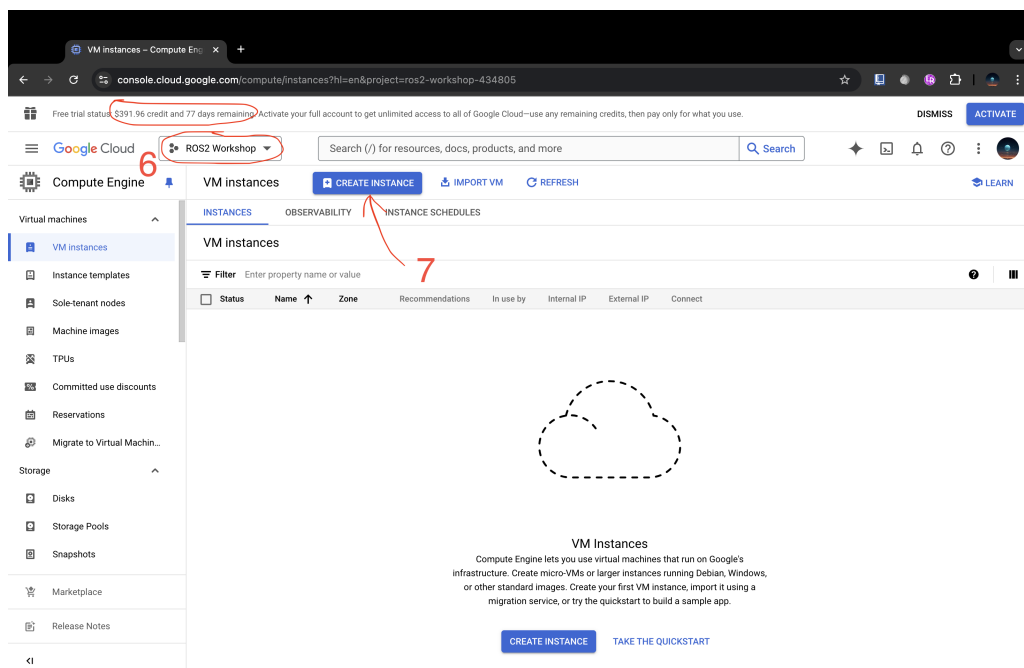
4. Name your project, no location is needed.



5. It will ask you to enable Compute Engine API so you can create VMs from your Google Cloud account.



6. At the top left, make sure you are in the project you want to work in. Note that the bar on the top lets you know how many credits you have and the time remaining on your trial.
7. Click 'Create Instance' and let's proceed to setting up your first VM!



8. Name your VM something helpful, set the region to 'us-central1 (Iowa)', and choose the C3 series for the machine type. Technically any region would be fine, but the US is generally cheaper and so saves us a few credits and has access to more machine types than Australia. The only downside is higher latency.

9. In the panel to the right, you can see a sort of 'shopping cart' that estimates roughly how much the current configuration will cost you in credits per hour and monthly. Note that the monthly rate is calculated assuming you will be running the VM 24/7.

Free trial status: \$391.96 credit and 77 days remaining. Activate your full account to get unlimited access to all of Google Cloud—use any remaining credits, then pay only for what you use.

Google Cloud ROS2 Workshop Search (/) for resources, docs, products, and more

Create an instance CREATE VM FROM... EQUIVALENT CODE

New VM instance
Create a single VM instance from scratch

New VM instance from template
Create a single VM instance from an existing template

New VM instance from machine image
Create a single VM instance from an existing machine image

Marketplace
Deploy a ready-to-go solution onto a VM instance

Name * ubuntu22-04-ros2

MANAGE TAGS AND LABELS

Region * us-central1 (Iowa)
Region is permanent

Zone * Any
Google will choose a zone on your behalf, maximizing VM obtainability. Zone is permanent.

Machine configuration

General purpose Compute optimized Memory optimized Storage optimized GPUs

Machine types for common workloads, optimized for cost and flexibility

Series	Description	vCPUs	Memory	Platform
☐ C4	Consistently high performance	2 - 192	4 - 1,488 GB	Intel Emerald Rapids
☐ N4	Flexible & cost-optimized	2 - 80	4 - 640 GB	Intel Emerald Rapids
☒ C3	Consistently high performance	4 - 192	8 - 1,536 GB	Intel Sapphire Rapids
☐ C3D	Consistently high performance	4 - 360	8 - 2,880 GB	AMD Genoa
☐ E2	Low cost, day-to-day computing	0.25 - 32	1 - 128 GB	Based on availability
☐ N2	Balanced price & performance	2 - 128	2 - 864 GB	Intel Cascade and Ice Lake
☐ N2D	Balanced price & performance	2 - 224	2 - 896 GB	AMD EPYC
☐ T2A	Scale-out workloads	1 - 48	4 - 192 GB	Ampere Altra Arm
☐ T2D	Scale-out workloads	1 - 60	4 - 240 GB	AMD EPYC Milan

Monthly estimate
\$295.35
That's about \$0.40 hourly
Pay for what you use: no upfront costs and per second billing

Item	Monthly estimate
8 vCPU + 32 GB memory	\$294.35
10 GB balanced persistent disk	\$1.00
Total	\$295.35

Compute Engine pricing

CREATE CANCEL EQUIVALENT CODE

10. Scroll down and select the specific machine type: 'c3-standard-8 (8 vCPU, 4 core, 32 GB memory)'. This should be enough to run our fairly intensive physics simulation.

Free trial status: \$391.96 credit and 77 days remaining. Activate your full account to get unlimited access to all of Google Cloud—use any remaining credits, then pay only for what you use.

Google Cloud ROS2 Workshop Search (/) for resources, docs, products, and more

Create an instance CREATE VM FROM... EQUIVALENT CODE

New VM instance
Create a single VM instance from scratch

New VM instance from template
Create a single VM instance from an existing template

New VM instance from machine image
Create a single VM instance from an existing machine image

Marketplace
Deploy a ready-to-go solution onto a VM instance

Machine type
Choose a machine type with preset amounts of vCPUs and memory that suit most workloads.

c3-standard-8 (8 vCPU, 4 core, 32 GB memory)

vCPU
8 (4 cores)

Memory
32 GB

ADVANCED CONFIGURATIONS

Availability policies
VM provisioning model: Standard
Choose "Spot" to get a discounted, preemptible VM. Otherwise, stick to "Standard". [Learn more](#)

VM PROVISIONING MODEL ADVANCED SETTINGS

CREATE CANCEL EQUIVALENT CODE

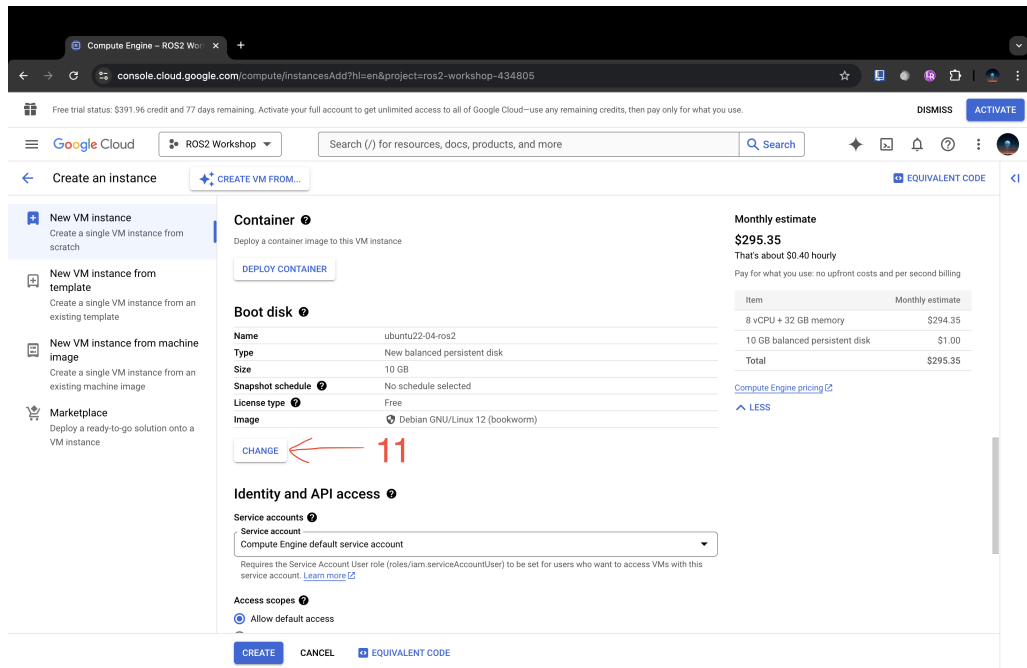
Monthly estimate
\$295.35
That's about \$0.40 hourly
Pay for what you use: no upfront costs and per second billing

Item	Monthly estimate
8 vCPU + 32 GB memory	\$294.35
10 GB balanced persistent disk	\$1.00
Total	\$295.35

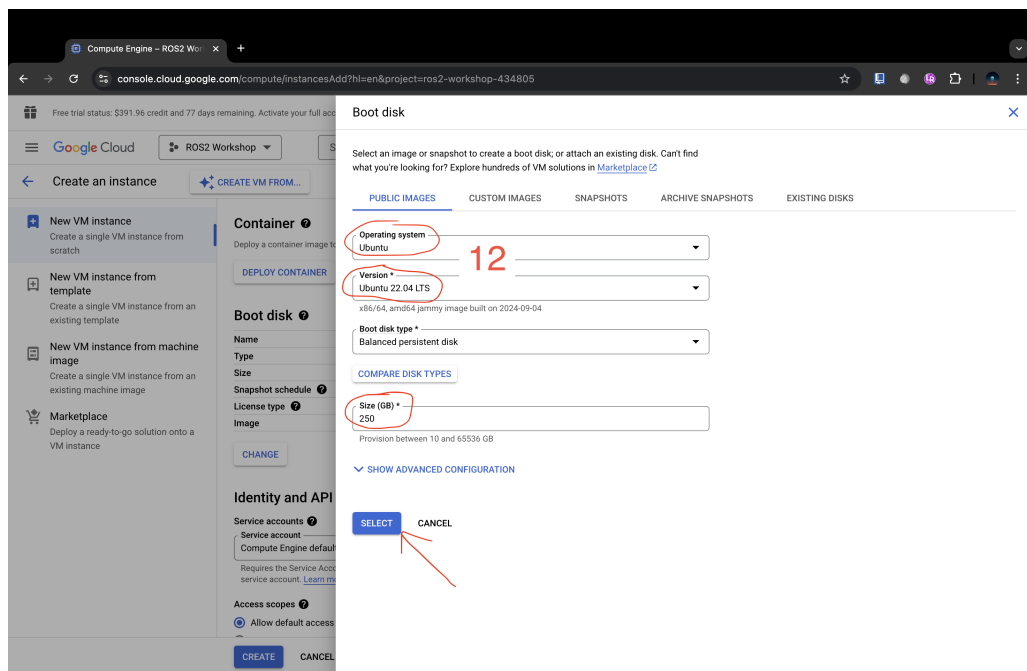
Compute Engine pricing

LESS

11. Scroll down further and change the boot disk so we can choose our OS and disk size.



12. Select Ubuntu, Ubuntu 22.04 LTS, and 250GB.



13. We are finished with our configuration, click 'EQUIVALENT CODE' to see what the equivalent commands would look like to set the VM up through Cloud Shell.

Free trial status: \$391.96 credit and 77 days remaining. Activate your full account to get unlimited access to all of Google Cloud—use any remaining credits, then pay only for what you use.

DISMISS ACTIVATE

Search (/) for resources, docs, products, and more

Create an instance CREATE VM FROM... EQUIVALENT CODE

New VM instance
Create a single VM instance from scratch

New VM instance from template
Create a single VM instance from an existing template

New VM instance from machine image
Create a single VM instance from an existing machine image

Marketplace
Deploy a ready-to-go solution onto a VM instance

Name * ubuntu22-04-ros2

MANAGE TAGS AND LABELS

Region * us-central1 (Iowa) Zone * Any

Region is permanent. Google will choose a zone on your behalf, maximizing VM obtainability. Zone is permanent.

Machine configuration

General purpose Compute optimized Memory optimized Storage optimized GPUs

Machine types for common workloads, optimized for cost and flexibility

Series	Description	vCPUs	Memory
☐ C4	Consistently high performance	2 - 192	4 - 1,488
☐ N4	Flexible & cost-optimized	2 - 80	4 - 640
☒ C3	Consistently high performance	4 - 192	8 - 1,536
☐ C3D	Consistently high performance	4 - 360	8 - 2,880
☐ E2	Low cost, day-to-day computing	0.25 - 32	1 - 128
☐ N2	Balanced price & performance	2 - 128	2 - 864

Monthly estimate
\$319.35
That's about \$0.44 hourly
Pay for what you use: no upfront costs and per second billing

Item	Monthly estimate
8 vCPU + 32 GB memory	\$294.35
250 GB balanced persistent disk	\$25.00
Total	\$319.35

Compute Engine pricing < LESS

Equivalent code

```

1 gcloud compute instances create
2   ubuntu22-04-ros2 \
3     --project=ros2-workshop-434805 \
4     --zone=us-central1-c \
5     --machine-type=c3-standard-8 \
6     --network-interface=network-tier=PREMIUM,
7       nic-type=GVNIC, stack-type=IPV4_ONLY,
8       subnet=default \
9     --maintenance-policy=MIGRATE \
10    --provisioning-model=STANDARD \
11    --service-account=346974516824-compute@developer.gserviceaccount.com \
12    --scopes=https://www.googleapis.com/auth/devstorage.read_only,https://www.googleapis.com/auth/logging.write,https://www.googleapis.com/auth/monitoring.write,https://www.googleapis.com/auth/service.management.readonly,https://www.googleapis.com/auth/servicecontrol,https://www.googleapis.com/auth/trace.append \
13    --create-disk=auto-delete=yes,boot=yes,device-name=ubuntu22-04-ros2,image=projects/ubuntu-os-cloud/global/images/ubuntu-2204-jammy-v20240904,mode=rw,size=250
14    type=pd-balanced \
15    --no-shielded-secure-boot \
16    --shielded-vtpm \
17    --shielded-integrity \
18    --labels=goog-ec-src \
19    --reservation-affinity=any

```

COPY RUN IN CLOUD SHELL View gcloud reference <

14. Double check the cost estimate panel matches, then click "Create".

Free trial status: \$391.96 credit and 77 days remaining. Activate your full account to get unlimited access to all of Google Cloud—use any remaining credits, then pay only for what you use.

DISMISS ACTIVATE

Search (/) for resources, docs, products, and more

Create an instance CREATE VM FROM... EQUIVALENT CODE

New VM instance
Create a single VM instance from scratch

New VM instance from template
Create a single VM instance from an existing template

New VM instance from machine image
Create a single VM instance from an existing machine image

Marketplace
Deploy a ready-to-go solution onto a VM instance

Name * ubuntu22-04-ros2

MANAGE TAGS AND LABELS

Region * us-central1 (Iowa) Zone * Any

Region is permanent. Google will choose a zone on your behalf, maximizing VM obtainability. Zone is permanent.

Machine configuration

General purpose Compute optimized Memory optimized Storage optimized GPUs

Machine types for common workloads, optimized for cost and flexibility

Series	Description	vCPUs	Memory	Platform
☐ C4	Consistently high performance	2 - 192	4 - 1,488 GB	Intel Emerald Rapids
☐ N4	Flexible & cost-optimized	2 - 80	4 - 640 GB	Intel Emerald Rapids
☒ C3	Consistently high performance	4 - 192	8 - 1,536 GB	Intel Sapphire Rapids
☐ C3D	Consistently high performance	4 - 360	8 - 2,880 GB	AMD Genoa
☐ E2	Low cost, day-to-day computing	0.25 - 32	1 - 128 GB	Based on availability
☐ N2	Balanced price & performance	2 - 128	2 - 864 GB	Intel Cascade and Ice Lake
☐ N2D	Balanced price & performance	2 - 224	2 - 896 GB	AMD EPYC
☐ T2A	Scale-out workloads	1 - 48	4 - 192 GB	Ampere Altra Arm
☐ T2D	Scale-out workloads	1 - 60	4 - 240 GB	AMD EPYC Milan

Monthly estimate
\$319.35
That's about \$0.44 hourly
Pay for what you use: no upfront costs and per second billing

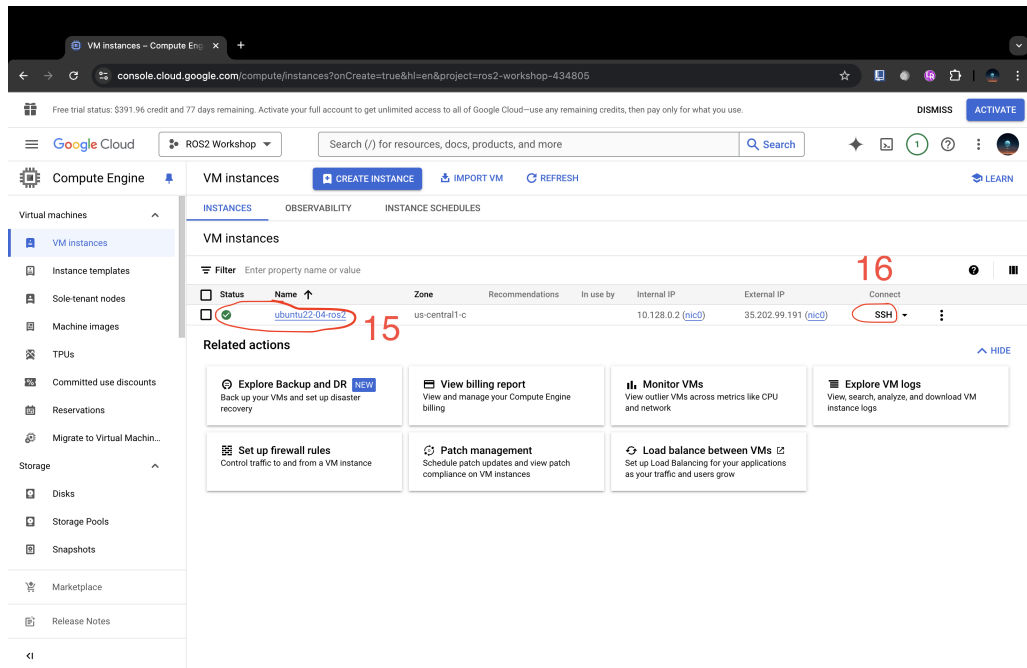
Item	Monthly estimate
8 vCPU + 32 GB memory	\$294.35
250 GB balanced persistent disk	\$25.00
Total	\$319.35

Compute Engine pricing < LESS

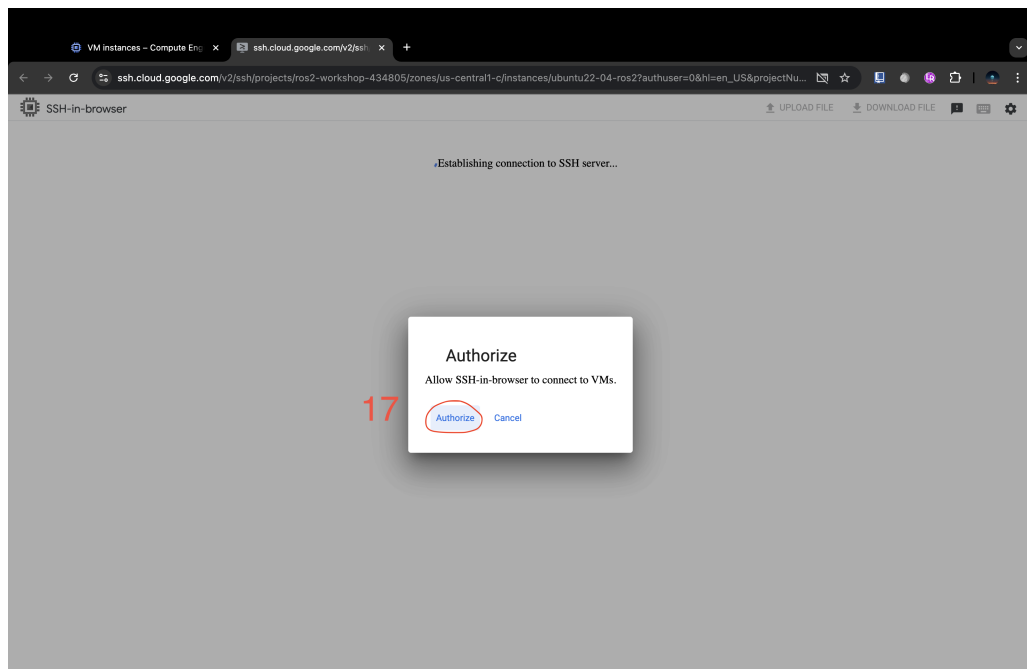
CREATE CANCEL EQUIVALENT CODE

15. Google Cloud will return you to the VM Instances page where you can see your instance being created. Wait until you see the green tick, which means it is done and active (starting to charge credits).

16. Click here to enter an in-browser SSH terminal for your VM.



17. Authorise the SSH terminal to connect to this VM. It will take a few seconds before the terminal appears.



18. From the terminal, we will now set up your user account with a password and sudo permissions for working on the remote desktop. Type:

```
sudo usermod -aG sudo <username>
```

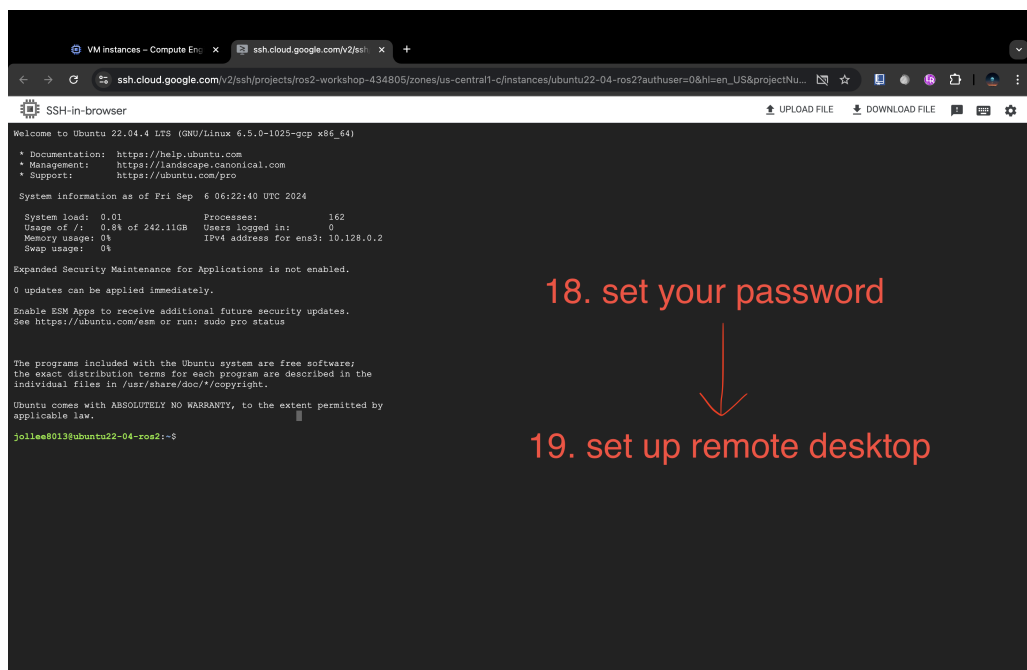
to add your username to the sudo list. For reference, the green text in terminal is <username>@<devicename>. After this, set your password by typing:

```
sudo passwd <username>
```

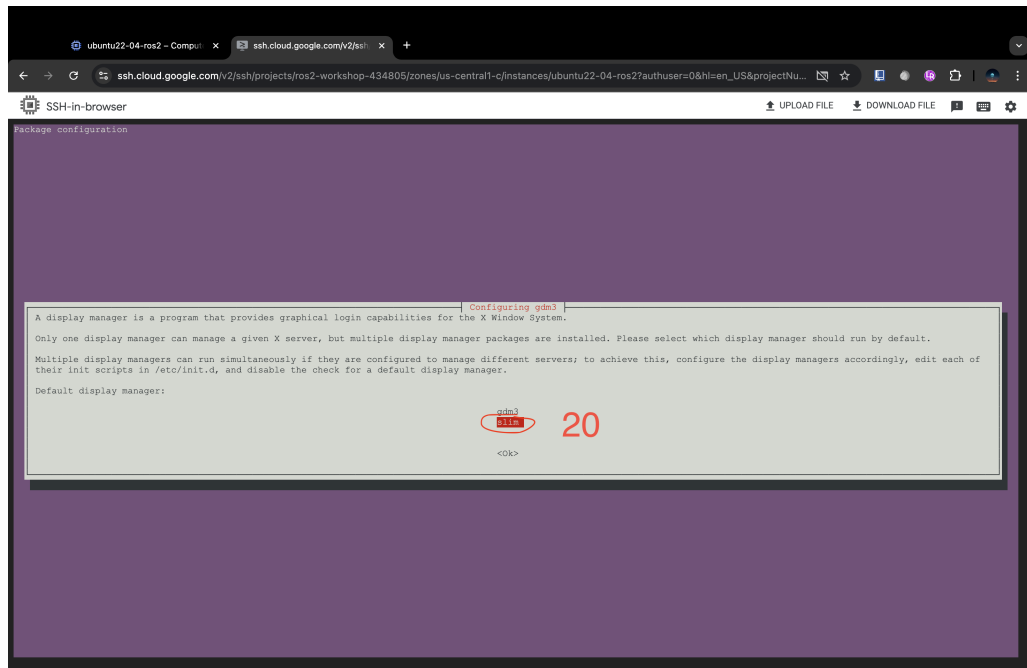
Don't forget your password! We will need it for later.

19. To set up the remote desktop so we have a GUI instead of just a terminal, run these commands:

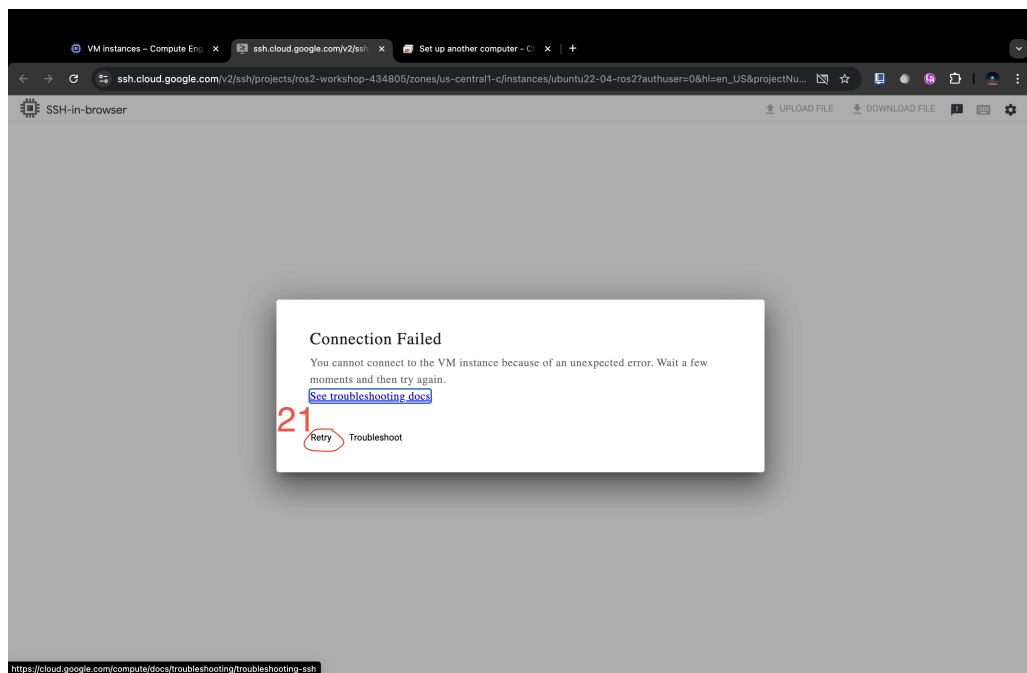
```
wget https://dl.google.com/linux/direct/chrome-remote-desktop\_current\_amd64.deb
sudo apt-get install --assume-yes ./chrome-remote-desktop\_current\_amd64.deb
sudo apt update && sudo apt upgrade
sudo apt install slim
sudo apt install ubuntu-desktop
```



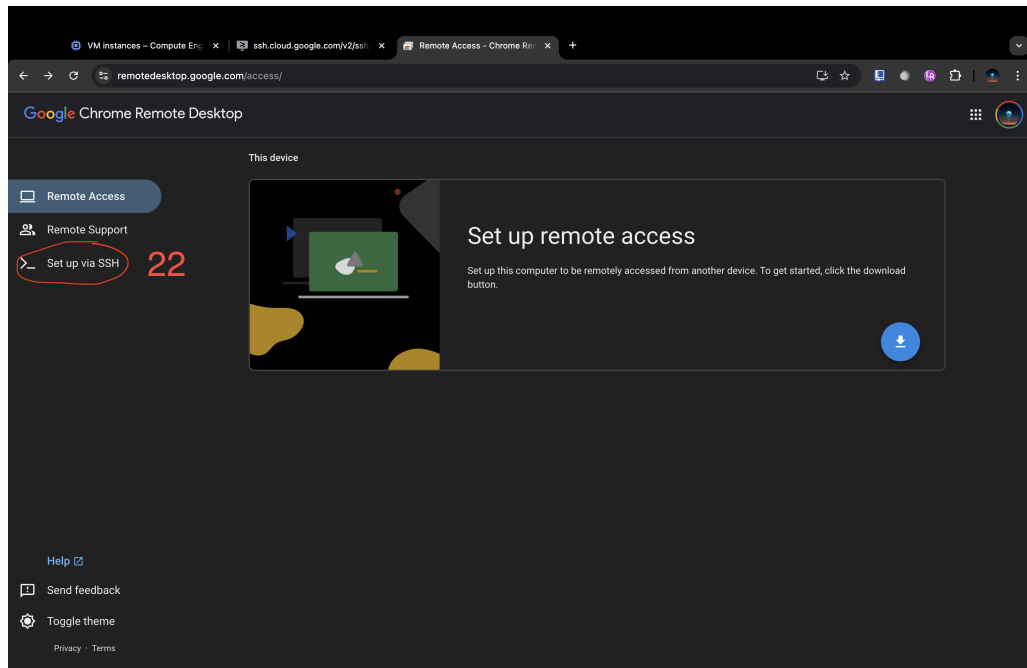
20. Press enter to choose slim as the display manager.



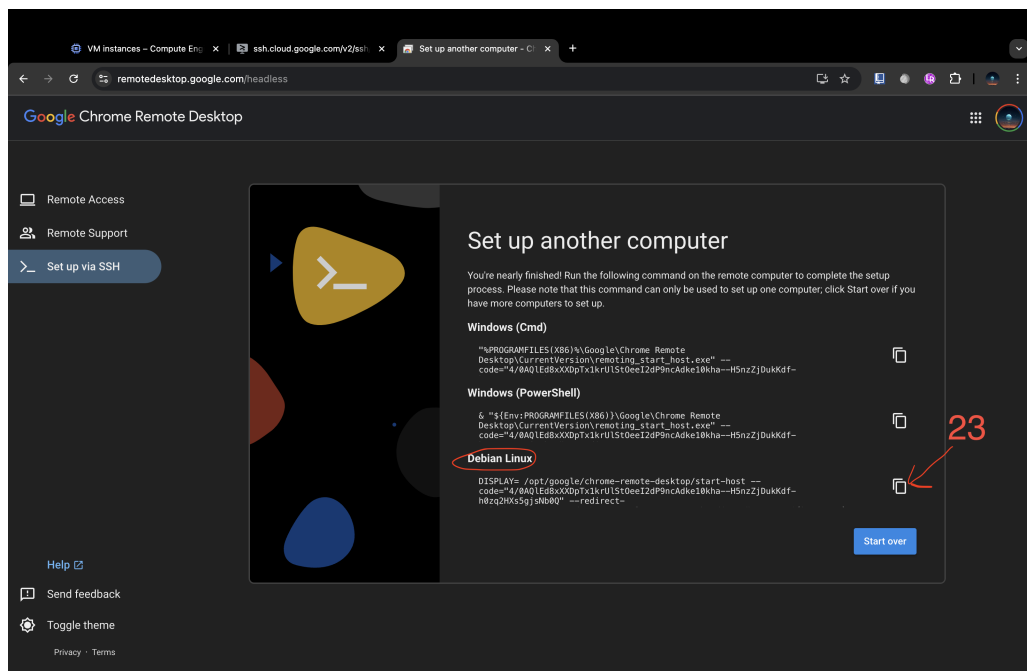
21. Reboot the VM with `sudo reboot` to affect all these changes. You will lose connection here, don't worry just reconnect again with SSH by clicking 'Retry'. Then run `sudo service slim start` and your VM will be ready to connect to a remote desktop.



22. Go to <https://remotedesktop.google.com/>. Here we will set up an in-browser remote desktop. Click on the "Set up via SSH" tab.



23. Follow the instructions until you arrive at this screen. Copy the command for Debian Linux.

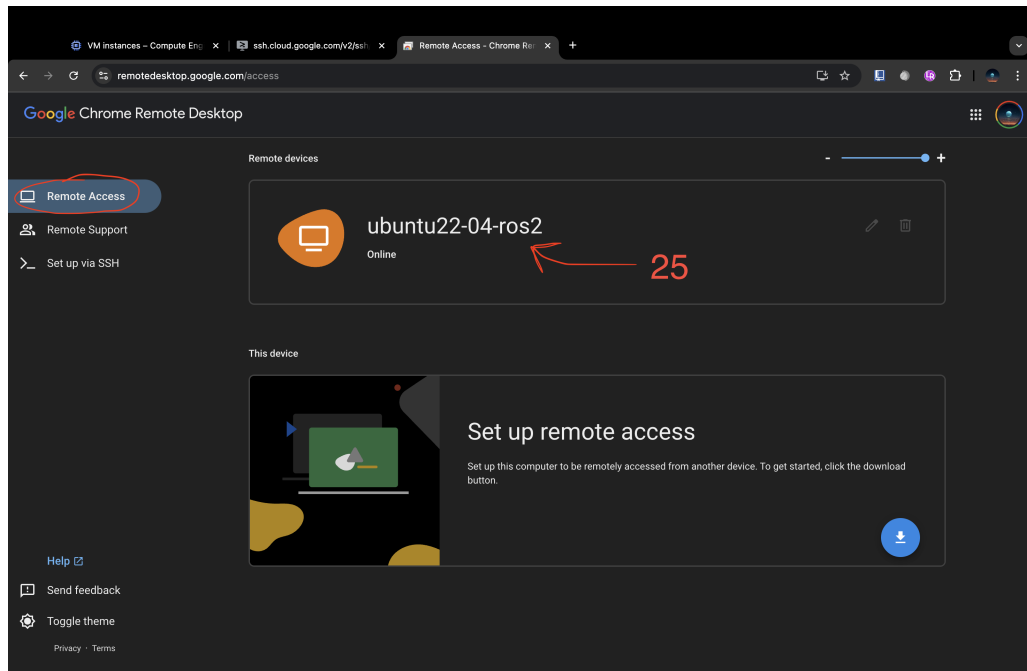


24. Run the command back in the SSH in-browser terminal and create a PIN number. Make sure you remember this! It should say after that the Host started successfully.

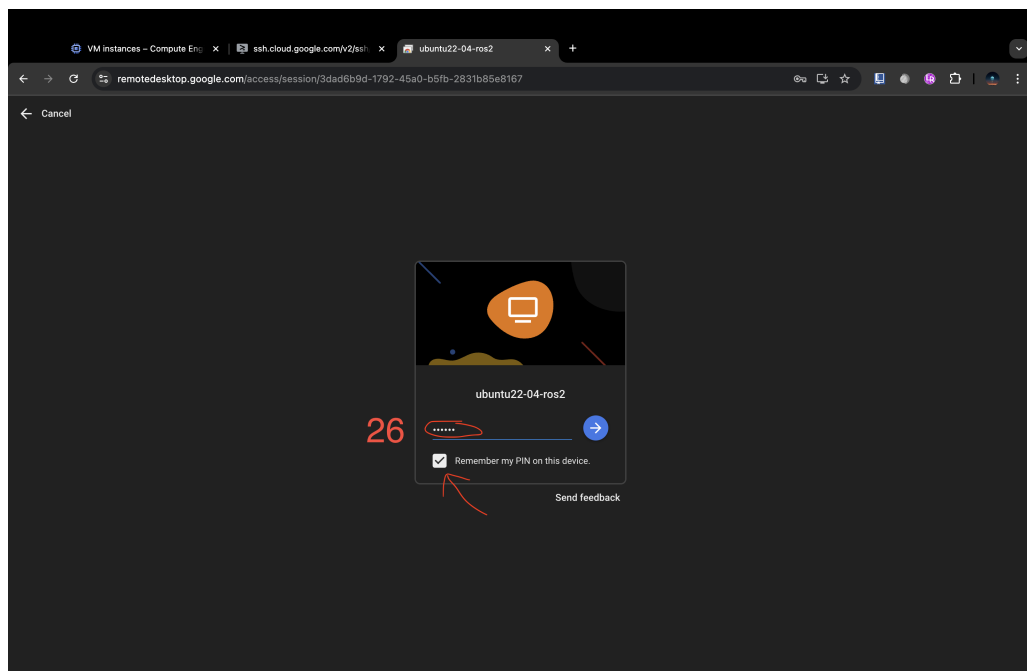
```
VM instances - Compute Eng... ssh.cloud.google.com/v2/ssh... Set up another computer - C...  
ssh.cloud.google.com/v2/ssh/projects/ros2-workshop-434805/zones/us-central1-c/instances/ubuntu22-04-ros2?authuser=0&hl=en_US&projectNu...  
SSH-in-browser  
jollie8013@ubuntu22-04-ros2:~$ DISPLAY= /opt/google/chrome-remote-desktop/start-host --code="4/OAQL8d8xEP_71U1-JOhSHQs3l63lpj0_x1un8ViuLd8gT51Y7TiFEZ4KQ6To4J5m3l6acp0" --redirect-url="https://remotedesktop.google.com/_/oauthredirect" --name=$(hostname)  
Enter a PIN of at least six digits:  
24
```

```
VM instances - Compute Eng... ssh.cloud.google.com/v2/ssh... Set up another computer - C...  
ssh.cloud.google.com/v2/ssh/projects/ros2-workshop-434805/zones/us-central1-c/instances/ubuntu22-04-ros2?authuser=0&hl=en_US&projectNu...  
SSH-in-browser  
jollie8013@ubuntu22-04-ros2:~$ DISPLAY= /opt/google/chrome-remote-desktop/start-host --code="4/OAQL8d8xEP_71U1-JOhSHQs3l63lpj0_x1un8ViuLd8gT51Y7TiFEZ4KQ6To4J5m3l6acp0" --redirect-url="https://remotedesktop.google.com/_/oauthredirect" --name=$(hostname)  
Enter a PIN of at least six digits:  
Enter the same PIN again:  
[0906/063305.207008:ERROR:host_config.cc(83)] Failed to read /home/jollie8013/.config/chrome-remote-desktop/host#588e48ac46a2719b3f4521857190462.json  
[0906/063306.042654:INFO:daemon_controller_delegate_linux.cc(98)] Created symlink /etc/systemd/system/multi-user.target.wants/chrome-remote-desktop@jollie8013.service -> /lib/systemd/system/chrome-remote-desktop.service.  
Host started successfully  
jollie8013@ubuntu22-04-ros2:~$
```

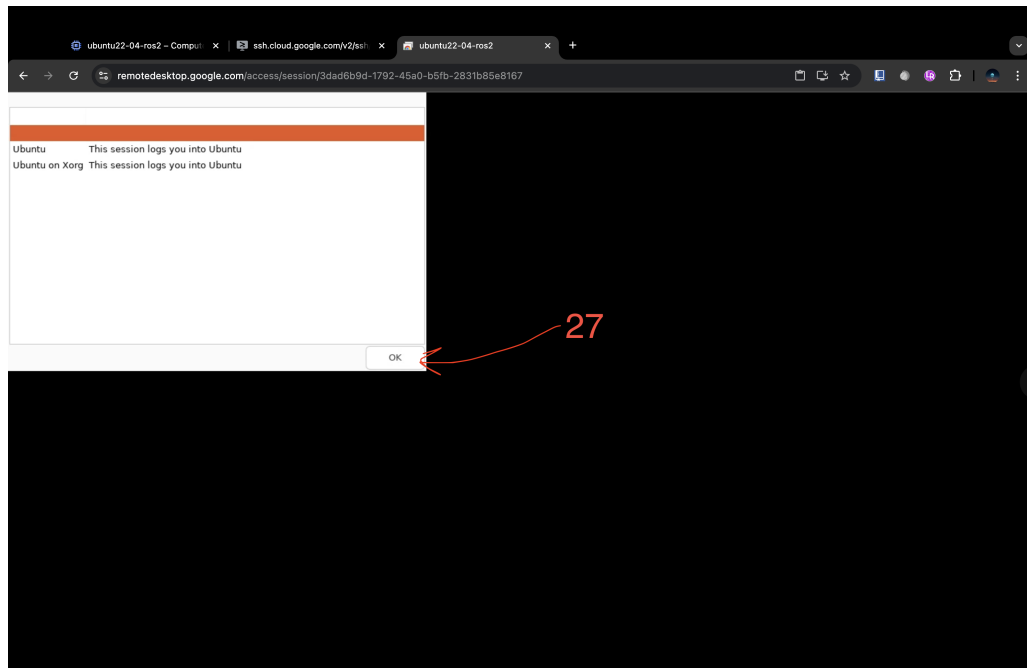
25. Go back to remotedesktop.google.com, your VM should be online now! Open it.



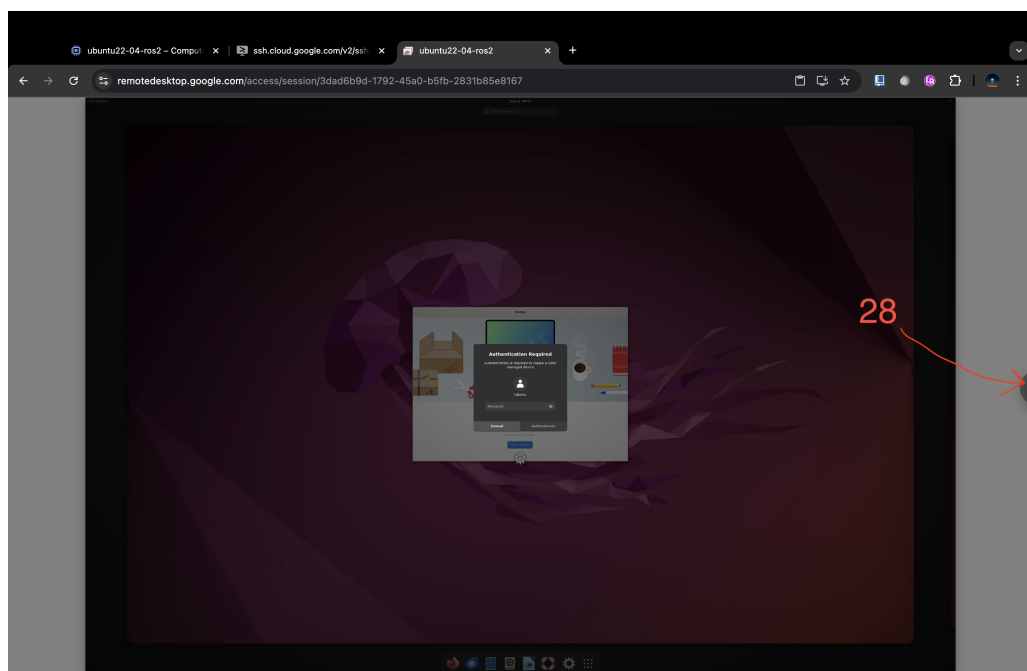
26. Enter your PIN and check the box to remember it for next time.



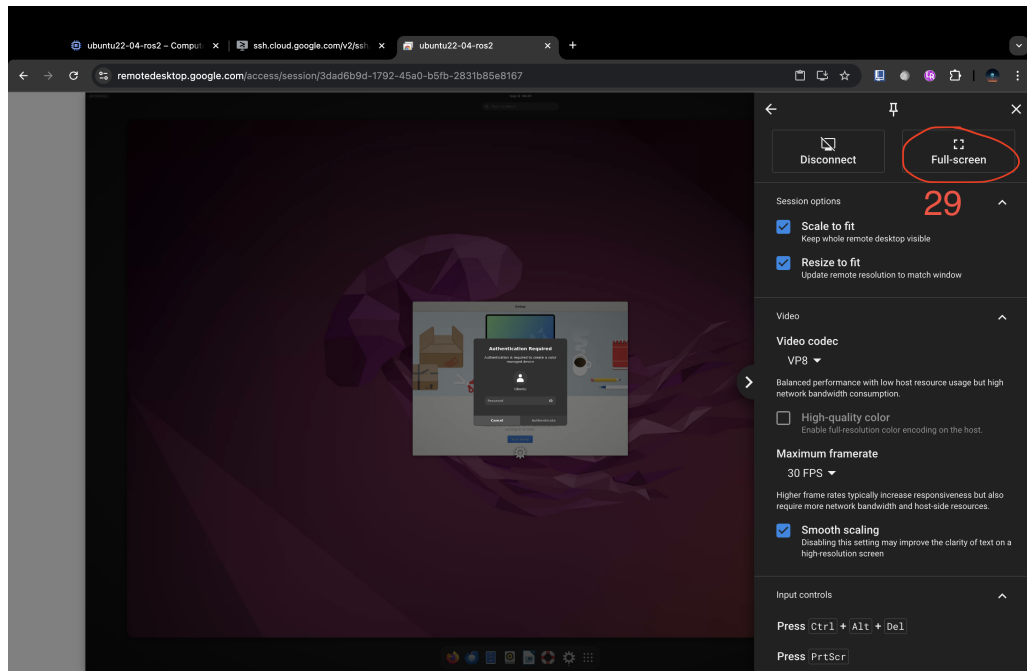
27. Ignore this window, press OK.



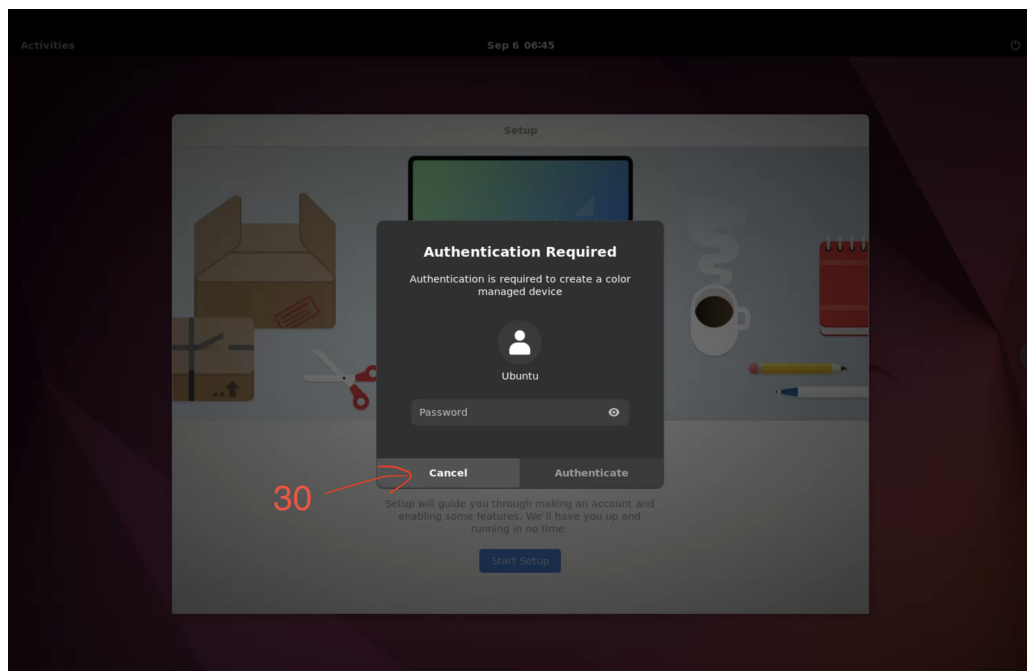
28. You should finally be able to see your VM's desktop. But the resolution will probably look a little off... click the arrow to open the sidebar menu.



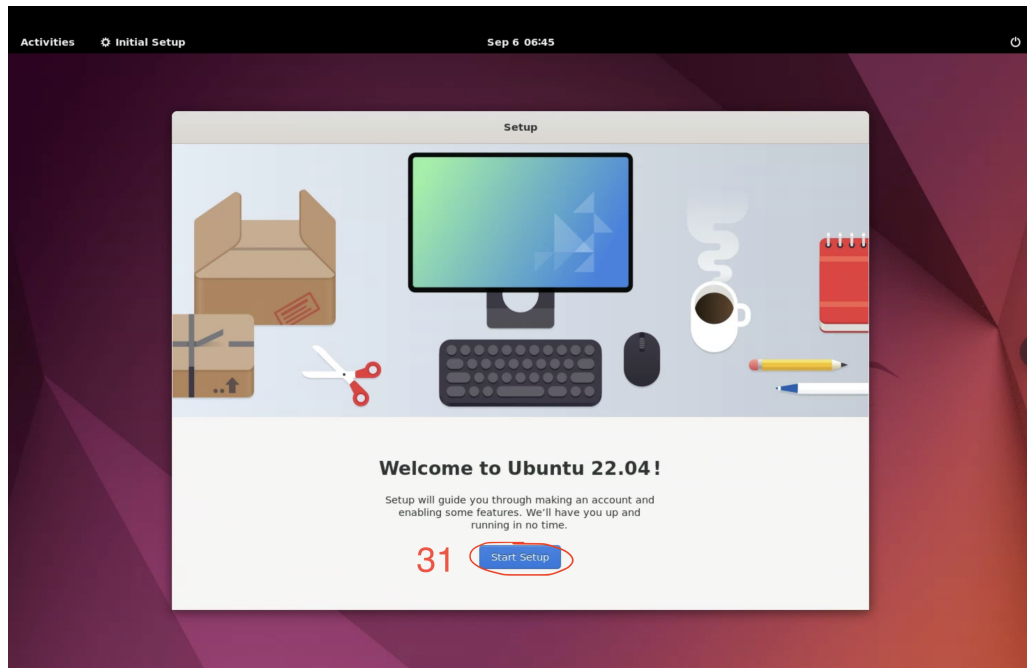
29. Select 'Full-screen' to re-scale the desktop.



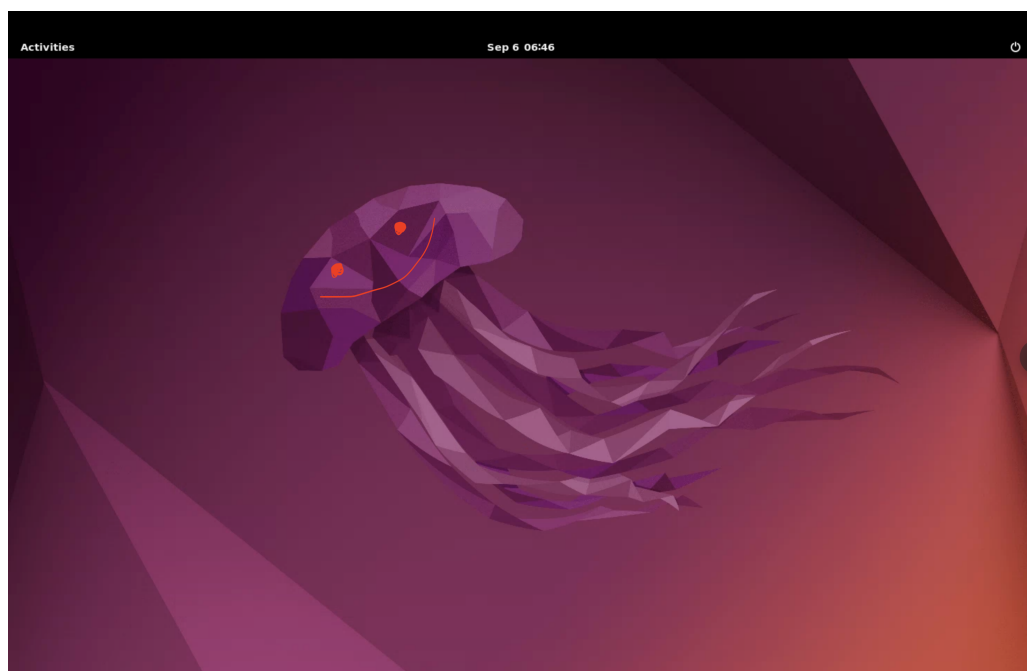
30. That's much better now! Ignore the authentication prompt, click cancel.



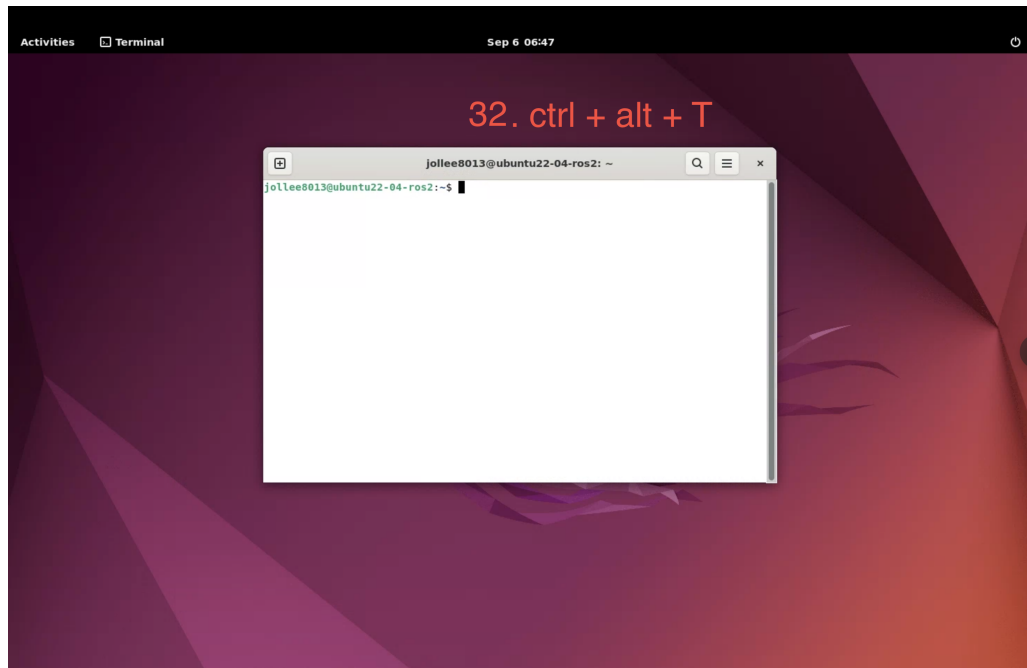
31. Quickly skip through the welcome window.



Ta-da! You have set up your very own 22.04 Jammy Jellyfish Ubuntu Virtual Machine! Let's go through some useful tools and install everything we need for the ROS workshop.

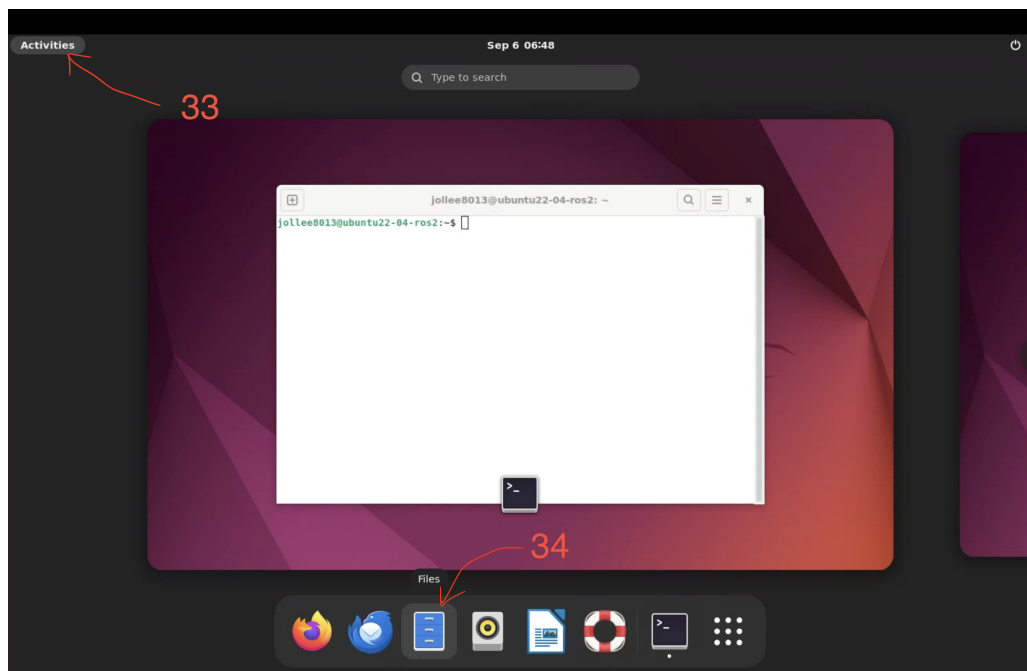


32. 'Ctrl + alt + T' (or 'Ctrl + option + T' on Mac) is a nice shortcut to open a new Terminal window to run commands in. We won't need the other in-browser SSH terminal tab anymore now that we have this, so you can close that.



33. You can also click 'Activities' on the top left show the dock at the bottom.

34. Click on 'Files'.



35. Here we can see the Home directory and other directories, much like any other computer. Clone this directory into home to get started with the installation

```
git clone https://github.com/ShivVary/ROSWorkshop.git
```

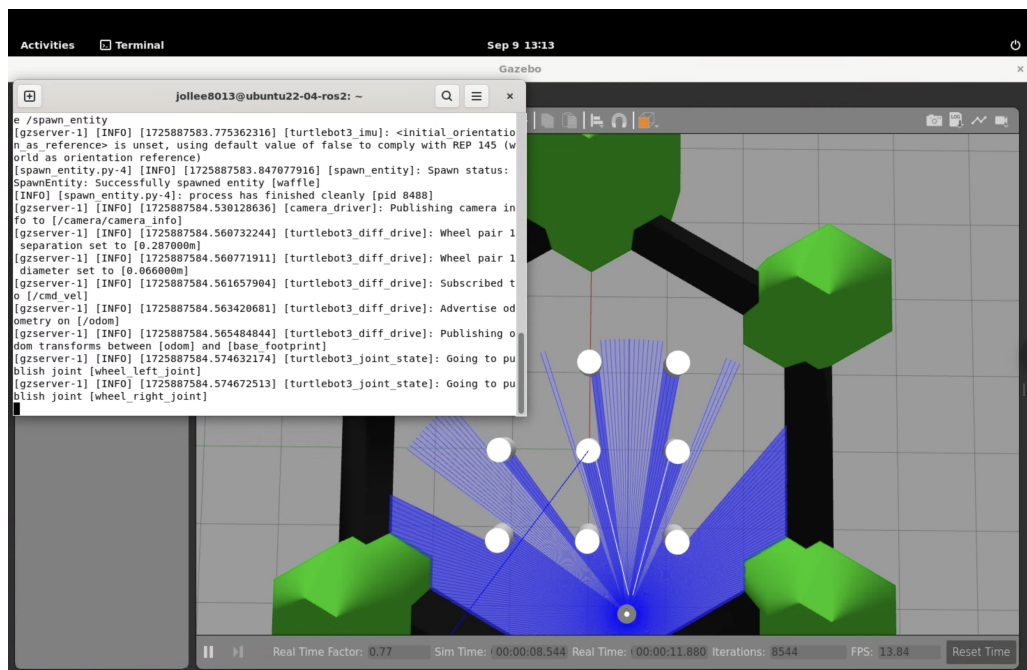
Change the directory to **Installation Files** and run these commands to install the ROS2, Gazebo and Turtlebot3 packages, as it calls the other three scripts:


```
chmod +x install_combo.sh
./install_combo.sh
```

36. You will now see it going through and installing everything. Follow any prompts if they appear to approve the installations. Wait for the installation to finish and then you're ready to go!
37. Once it is done, test that it works by running

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

It should look like the following:



If it does, congratulations you did everything right! If not, just let one of the friendly mentors know and they can help you out.

2 Module 2: Introduction to ROS

What is ROS ?

ROS stands for Robot Operating System is an open source software development kit for robotics applications. ROS offers a standard software platform to developers across industries that will carry them from research and prototyping all the way through to deployment and production.

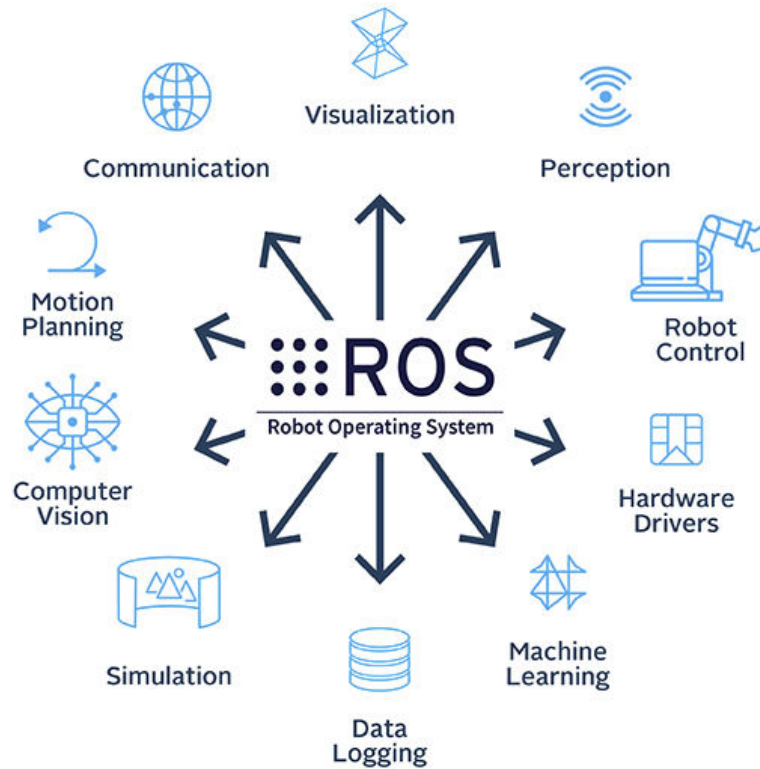


Figure 1

Many different fields and technology can be implemented with ROS to be deployed in robotics systems. Due to the complex behaviours robotic systems possess, ROS was developed to provide flexible framework for robotic software development that can robustly address the aforementioned issues.

ROS is released as distributions, also called “distros”, with more than one ROS distribution supported at a time. Some releases come with long term support (LTS), meaning they are more stable and have undergone extensive testing. Other distributions are newer with shorter lifetimes, but with support for more recent platforms and more recent versions of their constituent ROS packages.

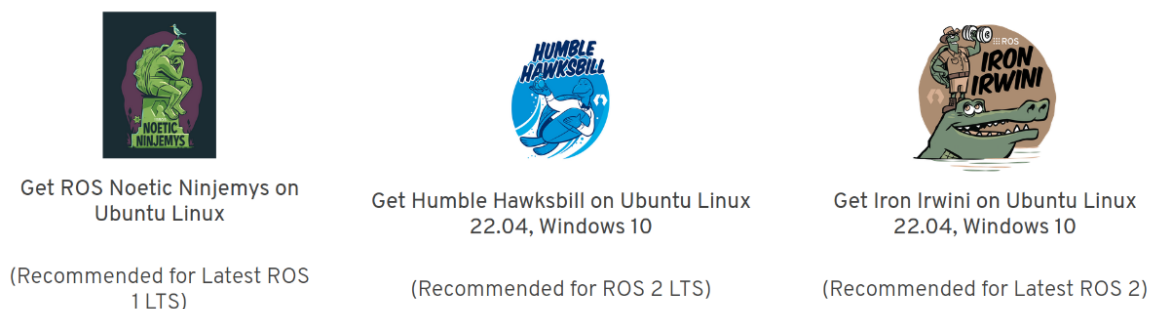


Figure 2

2.1 ROS 2 Humble Hawksbill

In this workshop, we will be using [ROS2 Humble Hawksbill](#). The link will take you to documentation page for all ROS 2 distros. The documentation will serve you well throughout this workshop so refer to it as much as possible when solving problems or searching for syntax. Here are the helpful links for documentations:

- [ROS2 Humble Hawksbill](#)
- [ROS 2 Humble Index](#)
- [Python 3 Documentations](#)
- [ROS Resources statuses](#)
- [ROS robots showcase](#)

2.2 Introductory exercises for ROS2

First let get you familiar with some common ROS commands. Type in this command in your linux terminal.

```
ros2 -h
```

You should get an output like this:

If you have encountered an error, it is likely you have not sourced the setup.bash file. For every new terminal you open, you will have to source it. Run this command:

```
source /opt/ros/humble/setup.bash
```

However, there is a smart way to get around this by writing the command above into your "bash.rc" file, basically your shell start up script. From your terminal run these commands:

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

2.3 Baby steps

First let install **turtlesim** if you have not already.

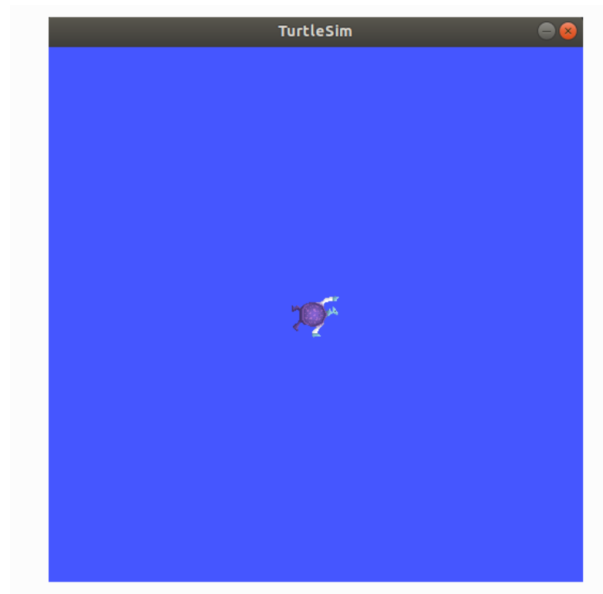
```
sudo apt update
sudo apt install ros-humble-turtlesim

# check if packages are installed
ros2 pkg executables turtlesim
```

Run the package

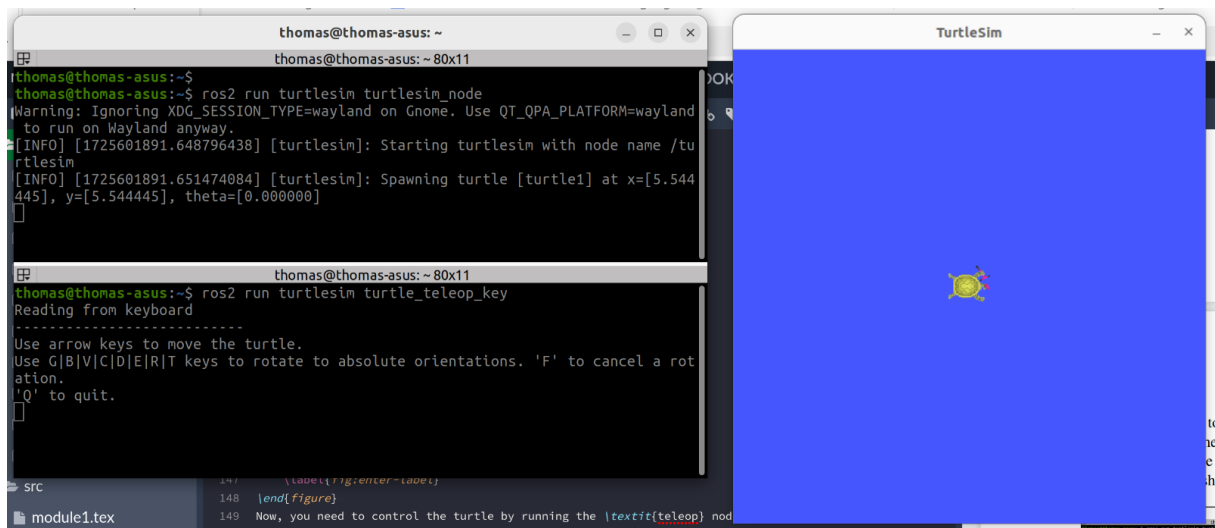
```
ros2 run turtlesim turtlesim_node
```

You should be able to see a pop up like this



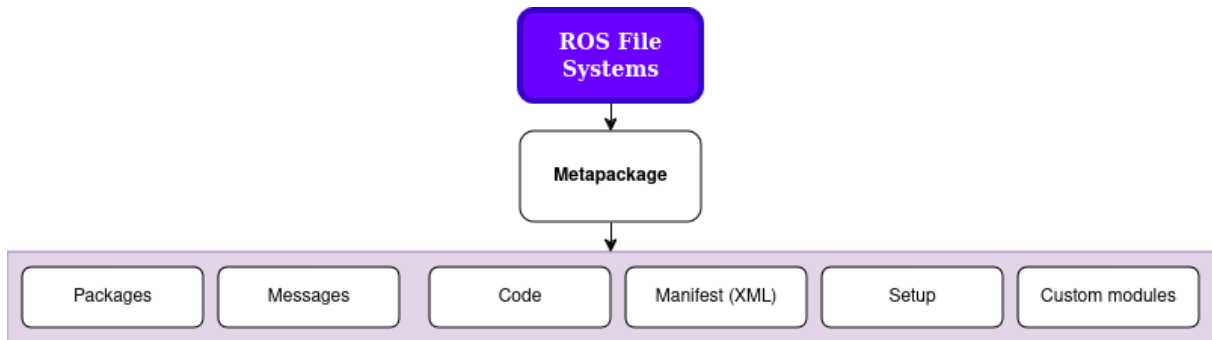
Now, you need to control the turtle by running the *teleop* node. You will need to open another terminal. We assume that you have installed the app *Terminator* which allows you to have multiple terminals opened one one window. Having this app and knowing its [shortcuts](#) can save you a lot of time. With that said, you should have an output like this screenshot once you entered this command in the new terminal:

```
ros2 run turtlesim turtle_teleop_key
```



Enough hand holding, from here on, follow this [tutorial link](#) to gain more insights on ROS network. We recommend that you complete it even if you have some knowledge about ROS2, it will serve as a good refresher. The linked tutorial will let you use RQT to visualise how nodes interact with each other to perform tasks. It will go over some concepts such as nodes, services and topics which will be touched with great details on in later sections.

3 Module 3: File structure



The file system level concepts mainly cover ROS resources that you encounter on disk, such as:

- **Packages:** Packages are the main unit for organizing software in ROS. A package may contain ROS runtime processes (nodes), a ROS-dependent library, datasets, configuration files, or anything else that is usefully organized together. Packages are the most atomic build item and release item in ROS. Meaning that the most granular thing you can build and release is a package.
- **Metapackages:** Metapackages are specialized Packages which only serve to represent a group of related other packages. Most commonly metapackages are used as a backwards compatible place holder for converted rosbuilt Stacks.
- **Package Manifests:** Manifests (package.xml) provide metadata about a package, including its name, version, description, license information, dependencies, and other meta information like exported packages. The package.xml package manifest is defined in REP-0127.
- **Repositories:** A collection of packages which share a common VCS system. Packages which share a VCS share the same version and can be released together using the catkin release automation tool bloom. Often these repositories will map to converted rosbuilt Stacks. Repositories can also contain only one package.
- **Message (msg) types:** Folder contain message descriptions define the data structures for messages sent in ROS. All message files end in **.msg**
- **Service (srv) types:** Folder contain service descriptions define the request and response data structures for services in ROS. Service files end in **.srv**
- **Action (action) types:** Folder contain service descriptions define the request and response data structures for action in ROS. Service files end in **.action**

Now that ROS2 concepts for file system is out of the way, we will walk you through how to translate all this into your ROS workshop. Run these commands in your terminal to let you visualise file [trees](#).

```
sudo apt-get install tree
```

3.1 Creating ROS2 workspace

Go to your home directory then create this a folder *ros2_ws*. Wonder these bash commands are for ?? Im not gonna walk you through this, check this link out for [linux shell commands](#).

```
# Create ROS WORKSPACE directory
cd ~
mkdir -p ~/ros2_ws/src

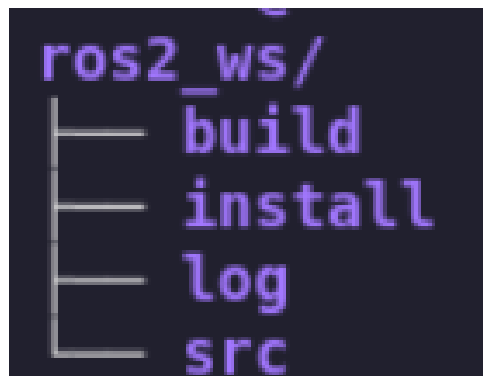
# GO TO SOURCE FOLDER OF WORKSPACE
cd ~/ros2_ws/src

# Get out of src
cd ..

# Build ur ros pacakges
colcon build

# Visulise your directories
tree -H ros2_ws ./ > ros.html
```

Click on the produced html file and have a look at your file structure. It should look similar like this depending on the command flag you put in your tree command.



If you encountered an issue where your shell does not recognise ROS2 commands, it is possible that you have not source your setup.bash properly. So back track to section where we show you [2.2](#).

3.2 ROS Packages (Python)

A package is an organizational unit for your ROS 2 code. If you want to be able to install your code or share it with others, then you'll need it organized in a package. With packages, you can release your ROS 2 work and allow others to build and use it easily.

Package creation in ROS 2 uses **ament** as its build system and colcon as its build tool. You can create a package using either CMake or Python, which are officially supported, though other build types do exist. Some packages even have cross languages build where they have both Python and C++ but it can be a hassle. For the time being, let's keep it simple.

Your packages are also **directories that must go into src folder of your workspace**. If your workspace does not follow conventional ROS2 file system, you may encounter some errors. Sometimes, it can work fine with the wrong structure but for good practice and to be safe, put them in your src folder.

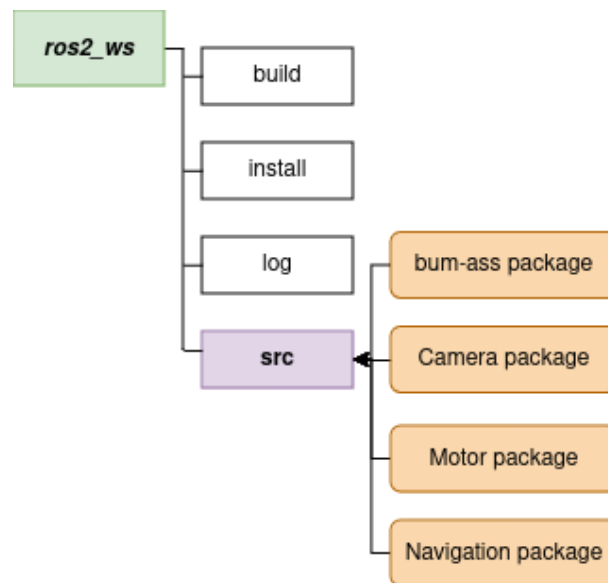
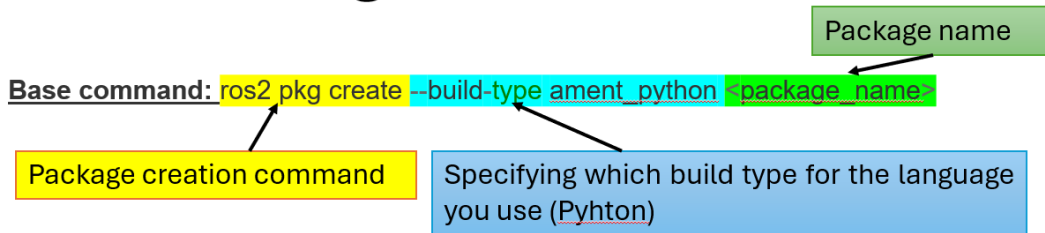


Figure 3: Packages organisation within **src**

3.3 Making your own ROS package

Otherwise follow these commands to create yourself a package within the src folder as well as a default node that prints out hello world. Within the **pkg create** command you can rename your node and package to something that you like as long as you follow [ROS naming conventions](#)

Ros2 Packages Creation



Ros2 Packages Creation

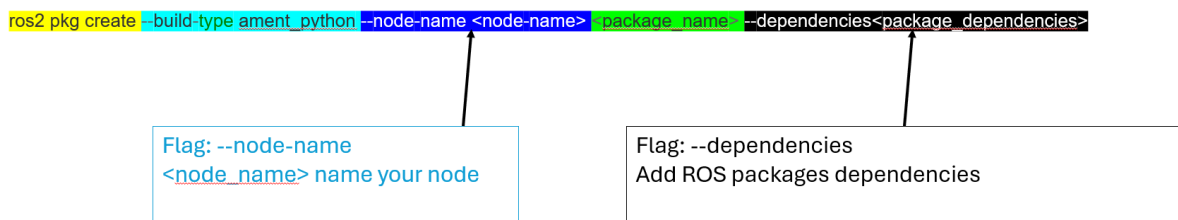


Figure 4

Adding different flags, defining your node name and declaring dependencies can help you automate modifications in the Package manifest and the subsequent files: `setup.py` and `setup.cfg`.

```
# Create ROS WORKSPACE directory from last steps
# U dont have to do this if ur already have a ros2 workspace
mkdir -p ~/ros2_ws/src

# Create a python packages with a node inside
# Replace my_node and my_package with whatever name you want but follow
# the naming conventions

cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python --license Apache-2.0 --node-name my_node my_package

# get out of ros2_ws/src
cd ..
colcon build

# You may need to source this to update the variable environment
source install/local_setup.bash

# OR
source ~/.bashrc
```



```
# Run your package node
ros2 run my_package my_node
```

You will notice that your package directory will contain these files

```
<package>    # whatever name u put down for ur package
LICENSE
<package>
  __init__.py
  <node>.py    # whatever name u put down for ur node
package.xml
resource
  <package>
setup.cfg
setup.py
test
  test_copyright.py
  test_flake8.py
  test_pep257.py
```

Now, looking at the file structures for the package we see different kinds of files: XML, .py, cfg folder. We will introduce you what these files are and show how to properly configure to your liking.

3.4 Python Packages: setup.py and setup.cfg

setup.py and **setup.cfg** are configuration files used for packaging and managing Python projects and since we are using Python in ROS2, these are needed. **Setup.py** script is used for define the vital information as follows: Metadata, packages, dependencies and Entry points (executable). **setup.cfg** is simply telling setuptools to put your executable in lib, because ros2 run will look for them there. Depending on the commands that you used for creating the packages, some part of the scripts are actually automated for you so you do not have to modify it after creation.

ROS 2 follows Python's standard module distribution process that uses setuptools. For Python packages, the setup.py file complements a C++ package's CMakeLists.txt. More details on distribution can be found in the [official documentation](#)..

```
import os
from glob import glob
from setuptools import setup

# name your package accordingly - to the same name of the directory of the package

package_name = 'my_package'

setup(
    name=package_name,
    version='0.0.0',
    # Packages to export
    packages=[package_name],
    # Files we want to install, specifically launch files
    data_files=[
        # Install marker file in the package index
        ('share/ament_index/resource_index/packages', ['resource/' + package_name]),
        # Include our package.xml file
        (os.path.join('share', package_name), ['package.xml']),
        # Include all launch files.
        (os.path.join('share', package_name, 'launch'),
```

```

        glob(os.path.join('launch', '*.launch.py'))),
    ],
    # This is important as well. Insert your details here
    # Shouldnt prevent you from running the code but... good practice otherwise
    install_requires=['setuptools'],
    zip_safe=True,
    author='ROS 2 Developer',
    author_email='ros2@ros.com',
    maintainer='ROS 2 Developer',
    maintainer_email='ros2@ros.com',
    keywords=['foo', 'bar'],
    classifiers=[
        'Intended Audience :: Developers',
        'License :: TODO',
        'Programming Language :: Python',
        'Topic :: Software Development',
    ],
    description='My awesome package.',
    license='TODO',
    # Like the CMakeLists add_executable macro, you can add your python
    # scripts here.

    # If you only created the package without specifying a node, you may need to modify this
    entry_points={
        'console_scripts': [
            '<my_script> = <my_package>.<my_script>:main'
        ],
    },
    },
)

```

Next up is **setup.cfg**, make sure that every `<package-name>` is the same name as what you named your package

```

[develop]
script_dir=$base/lib/<package-name>
[install]
install_scripts=$base/lib/<package-name>

```

3.5 Package XML

Package manifest file which marks the root of a package and contains meta information about the package including its name, version, description, maintainer, license, dependencies, and more. The contents of the manifest are in machine readable XML format and the contents are described in the REPs 127 and 140, with the possibility of further modifications in future REPs.

For ROS2, the documentation is still not as comprehensive as ROS 1 so I am pulling my hair out to search for proper documentations while doing Thesis B so excuse my yapping.

When you are defining dependencies, there are 5 types. Many of these carried over from ROS 1 Format:

- **Build Dependencies** : Specify which packages are needed to build this package. This is the case when any file from these packages is required at build time. `build_depend`
- **Build Export Dependencies** : Specify which packages are needed to build libraries against this package. `build_export_depend`
- **Execution Dependencies**: Specify which packages are needed to run code in this package. This is the case when you depend on shared libraries in this package. `exec_depend`

- **Test Dependencies:** specify only additional dependencies for unit tests. They should never duplicate any dependencies already mentioned as build or run dependencies. `test_depend`
- **Mixed dependencies** For mixed purposes, use `depend`, which covers build, export, and execution time dependencies.

See next page for the dependencies commands for ROS2 Colcon. Again, if you have created the packages

```
<?xml version="1.0"?>
<package format="3">
  <name>test-nodes</name>
  <version>0.0.0</version>
  <description>Test Nodes</description>

  <!-- One maintainer tag required, multiple allowed, one person per tag -->
  <!-- Example: -->
  <!-- <maintainer email="jane.doe@example.com">Jane Doe</maintainer> -->
  <maintainer email="matmur@amazon.com">matmur</maintainer>

  <!-- One license tag required, multiple allowed, one license per tag -->
  <!-- Commonly used license strings: -->
  <!-- BSD, MIT, Boost Software License, GPLv2, GPLv3, LGPLv2.1, LGPLv3 -->
  <license>TODO</license>

  <!-- Url tags are optional, but multiple are allowed, one per tag -->
  <!-- Optional attribute type can be: website, bugtracker, or repository -->
  <!-- Example: -->
  <!-- <url type="website">http://wiki.ros.org/beginner_tutorials</url> -->

  <!-- Author tags are optional, multiple are allowed, one per tag -->
  <!-- Authors do not have to be maintainers, but could be -->
  <!-- Example: -->
  <!-- <author email="jane.doe@example.com">Jane Doe</author> -->

  <build_depend>example_interfaces</build_depend>
  <build_export_depend>example1</build_export_depend>
  <test_depend>ament_copyright</test_depend>
  <test_depend>ament_flake8</test_depend>
  <test_depend>ament_pep257</test_depend>
  <test_depend>python3-pytest</test_depend>

  <!-- The export tag contains other, unspecified, tags -->
  <export>
    <!-- Other tools can request additional information be placed here -->
  </export>
</package>
```

4 Module 4: Computational Graph

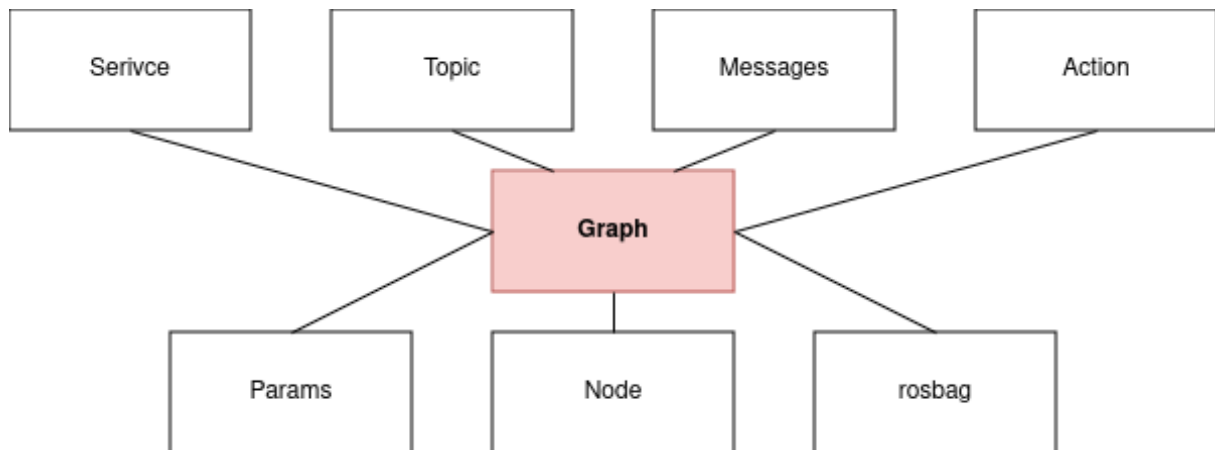


Figure 5

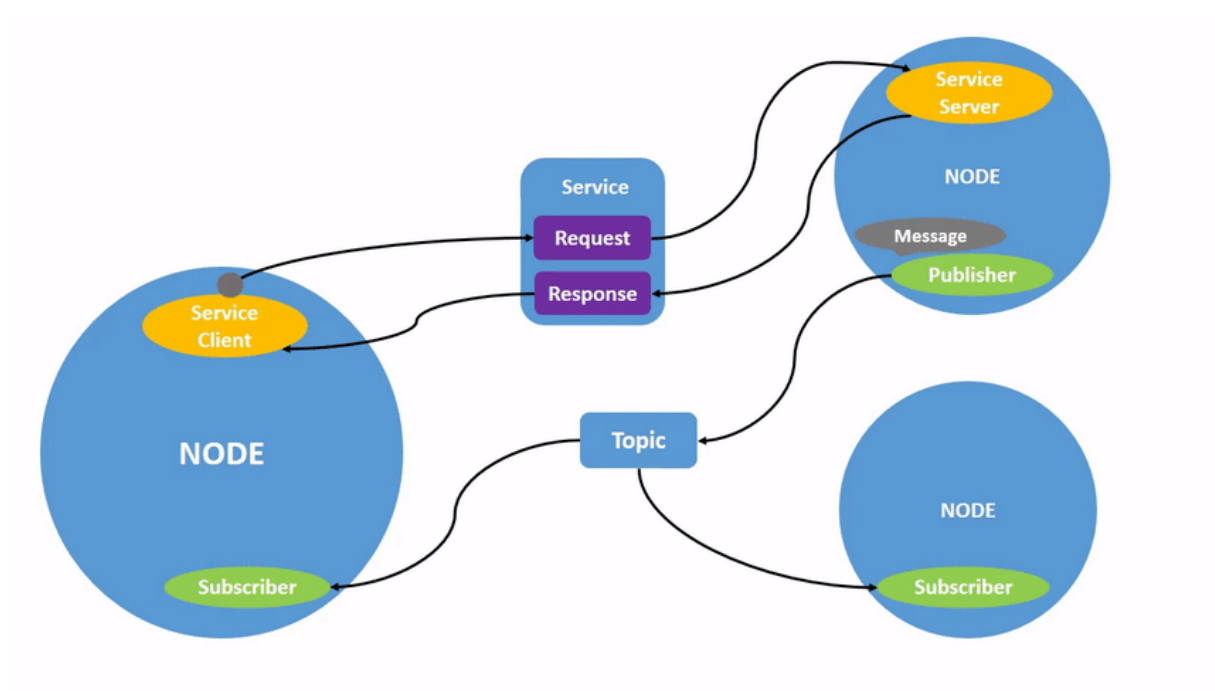


Figure 6: Communications between nodes

Watch these youtube links to get briefed on these ROS computational concepts.

- [Nodes,Parameters and Topics](#)
- [Services and Actions](#)

4.1 Nodes, Subscriber, Publisher, Topics and Messages

ROS NODES



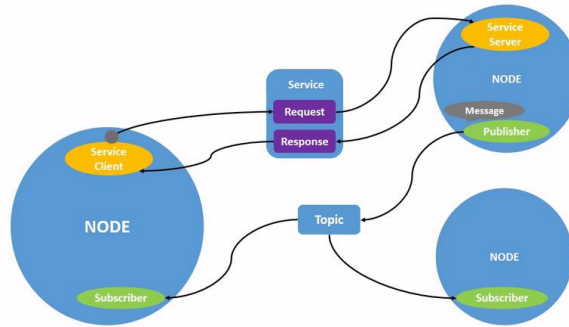
Process that performs computations like signal processing, computer vision, driving motors or localization...



An executable that performs the computations in a ROS network

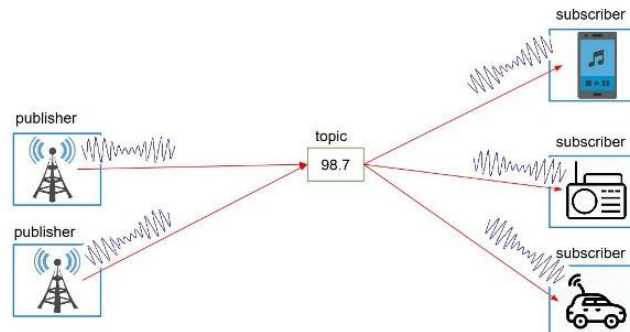


Imagine within a company, you have bunch of employees who do their assigned tasks – **Nodes are the employees.**



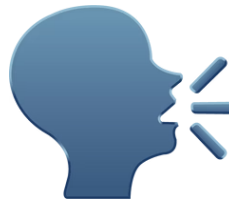
Ros Topics

- Think of it as a radio channel where a specific type of **message** can be broadcasted (published) and listened by others.
- Topics are a vital element of the ROS graph that act as a bus for nodes to exchange messages.
- Naming convention can affect accessibility



Publisher

Publishing refers to the action performed by a node to send data (messages) to a topic, making it available to any other nodes that are subscribed to that topic



Properties:

- Asynchronous
- Non blocking

```
<publisher_var>= <node_class>.create_publisher(<msg type>, '<topic name>', queue size)
```

Define message type

Invoke Node's class method to create a publisher

Define Queue determines the number of messages held temporarily in the queue before being processed or sent.– Why ??

Subscriber

Subscribing refers to the action performed by a node to receive data (messages) from a topic and process it if required.



```
<sub_var>= <node_class>.create_subscription(<msg type>, <topic>, <node_class>.call_back_function, queue_size)
```

Callback function processes the content of the message every time a message is received by the node

Questions

- Can a node do both subscribing and publishing ?
- For subscribers it is necessary to process data every time you receive a message ? If not, how do we define a call back function for it?
- Why does queue size matter in publishing and subscribing ?
- Does syntax of topics matter ?

Messages

.msg files that define data values for ROS nodes to publish.

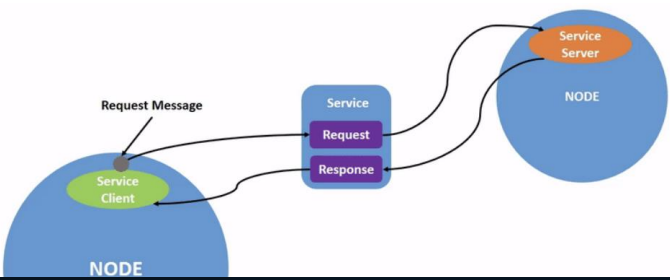
Format whatever data is being published or subscribed.

You can custom make your own .msg and define its own data by using the available built in data type declarations.

Primitive Type	Serialization	C++	Python2	Python3
bool (1)	unsigned 8-bit int	uint8_t (2)	bool	
int8	signed 8-bit int	int8_t	int	
uint8	unsigned 8-bit int	uint8_t	int (3)	
int16	signed 16-bit int	int16_t	int	
uint16	unsigned 16-bit int	uint16_t	int	
int32	signed 32-bit int	int32_t	int	
uint32	unsigned 32-bit int	uint32_t	int	
int64	signed 64-bit int	int64_t	long	int
uint64	unsigned 64-bit int	uint64_t	long	int
float32	32-bit IEEE float	float		float
float64	64-bit IEEE float	double		float
string	ascii string (4)	std::string	str	bytes
time	secs/nsecs unsigned 32-bit ints	ros::Time		rospy.Time
duration	secs/nsecs signed 32-bit ints	ros::Duration		rospy.Duration

4.2 Actions and Services

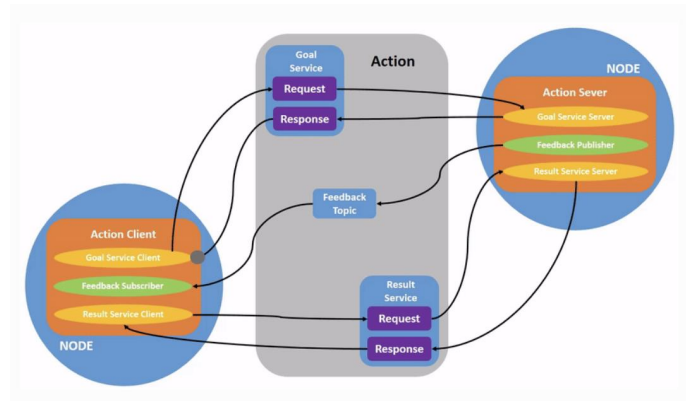
Services



- call-and-response model
- Blocking
- What is the suitable analogy for services in Ros ?

Action

- Actions are one of the communication types in ROS 2 and are intended for long running tasks. They consist of three parts: **a goal, feedback, and a result.**
- Their functionality are like services, except actions can be canceled and NON BLOCKING.



4.3 Basic Tutorials for all concepts

We recommend that you go to ROS2 Humble beginner tutorial links to understand the basic codes for nodes, subscribers, publishers and other concepts that were just introduced. You are more than welcome to walk through the entire Beginner CLI Tutorials and Beginner Client Tutorials from ROS Humble documentation if you want to be thorough. These main links will take you straight to the basic ROS concepts we have just mentioned.

- [Subscriber and Publisher](#)
- [Writing a simple service and client \(Python\)](#)
- [Creating custom msg and srv files](#)
- [Ros action in python](#)
- [Recording and running Bagfiles](#)

For each of the basic lesson, you should create your own packages to better organise them.

```
# Create package for publisher
ros2 pkg create --build-type ament_python --license Apache-2.0 --node-name pub_node publisher

# Create package for subscriber
ros2 pkg create --build-type ament_python --license Apache-2.0 --node-name sub_node subscriber

# Create package for Services
ros2 pkg create --build-type ament_python --license Apache-2.0 py_srvcli --dependencies rclpy
→ example_interfaces

# Create package for custom messages and service messages
ros2 pkg create --build-type ament_cmake --license Apache-2.0 tutorial_interfaces

# Create package for Action
ros2 pkg create --build-type ament_python --license Apache-2.0 action_interface
```


5 Module 5: Basic Exercises

5.1 Controlling Turtlebot in Gazebo

Now we going to get you to familiarise with simulations using Gazebo ad Rviz. In this exercise, you will be able to see how robots visualise the environment with Lidar and camera. In practice, these will be used as a way to collect data about the environment to make decisions. To launch the world and the turtlebot model, run these commands below:

```
export TURTLEBOT3_MODEL=waffle
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

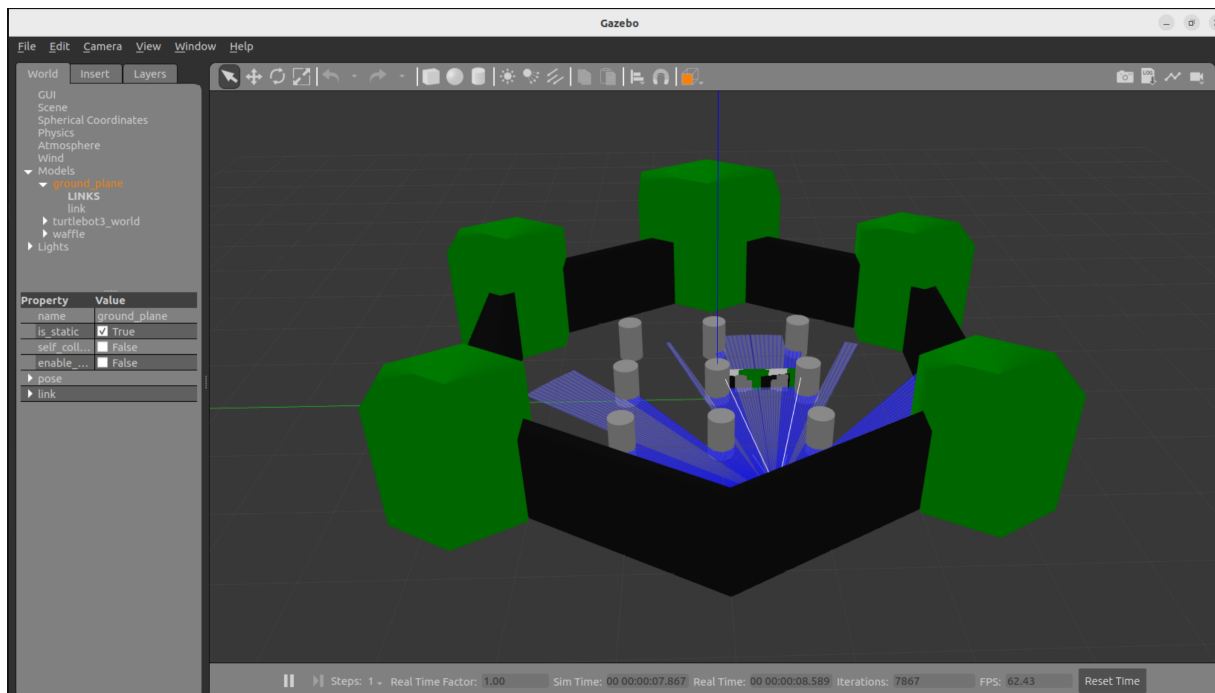


Figure 7: Output should look like this

In another terminal, launch the below command to visualise the Lidar Data and camera of the robot:

```
ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=True
```

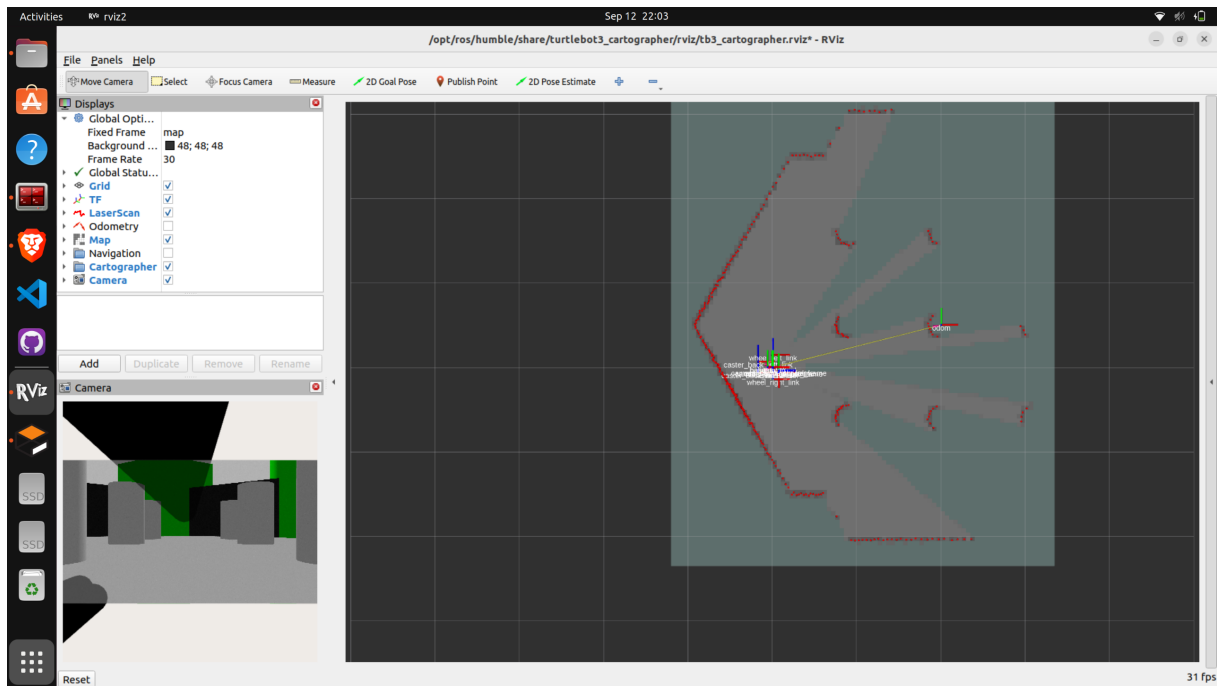


Figure 8: Rviz should look like this

In another terminal AGAIN, launch the below command so you can control the turtlebot within Gazebo with your keyboard:

```
ros2 run turtlebot3_teleop teleop_keyboard
```

To access the camera follow the steps - go to bottom left of Rviz and select Add. Then go to by topic can

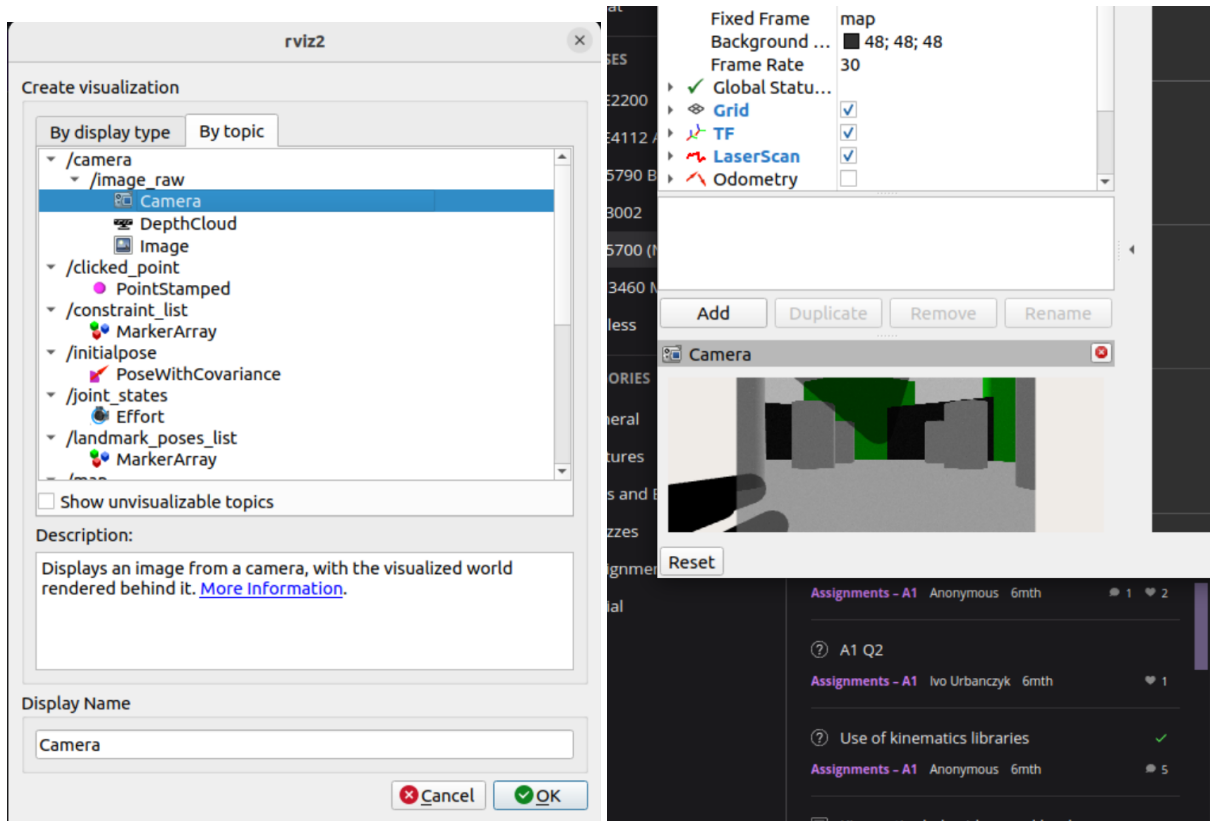


Figure 9: Caption

Go to this link to see how to save an actual map of the area once you have managed to map out everything with Lidar scans [turtlebot 3 SLAM simulations](#).

5.2 Publishers and Subscribers: PIG LATIN



Figure 10

Background

Pig Latin is a playful and altered form of English used primarily as a language game among children and friends. The main idea behind Pig Latin is to transform words in a way that sounds funny or coded, making it more challenging for outsiders to understand.

- **Words that begin with a consonant sound:**

- Move the first consonant or consonant cluster (group of consonants) to the end of the word.
- Add "ay" to the end of the resulting word.
- e.g. "hello" → "ello**hay**", "pig" → "ig**pay**", "string" → "ing**stray**"

- **Words that begin with a vowel sound (a, e, i, o, u):**

- Simply add "way" or "yay" to the end of the word (both versions are commonly accepted).
- e.g. "apple" → "apple**way**" or "apple**yay**"
- "orange" → "orange**way**" or "orange**yay**"

- **Handling words with no vowels (rare but possible):**

- The letter 'Y' creates a tricky situation when forming words in Pig Latin. Users have to determine whether the letter is a vowel or consonant based on its location in a name. Here are some essential facts to remember:
 1. When the letter 'Y' comes at the beginning of an English word, you consider it a consonant. This means you move it to the end of the term and then add suffix -ay. For instance:
 - * **Year** becomes eary+ay to form **earyay**
 - * **Yellow** becomes ellowy+ay to form **ellowyay**
 2. When the letter 'Y' appears in the second position of a word, the standard rules for word formation apply:
 - * **Dyke** becomes yked+ay to form **ykeday**
 3. If you have a consonant cluster at the beginning of the word and the letter 'Y' comes at the end of this cluster, you treat it as a vowel and hence doesn't move with the consonant cluster. Example:
 - * **Rhythm** becomes ythmrh+ay to form **ythmrhay**

- **Creating Pig Latin words from Compound Words :**

- You should first separate the compound word into individual words and apply the appropriate rules. For instance;
 - * **Bathroom** becomes **athbayoomray**
 - * **Toothbrush** becomes **oothtayushbray**

Task

Your task is to code a Publisher Node that ask for user input messages then publishes messages to a topic. Then a Subscriber Node subscribes to that topic and encode the messages and print them out to the terminal.

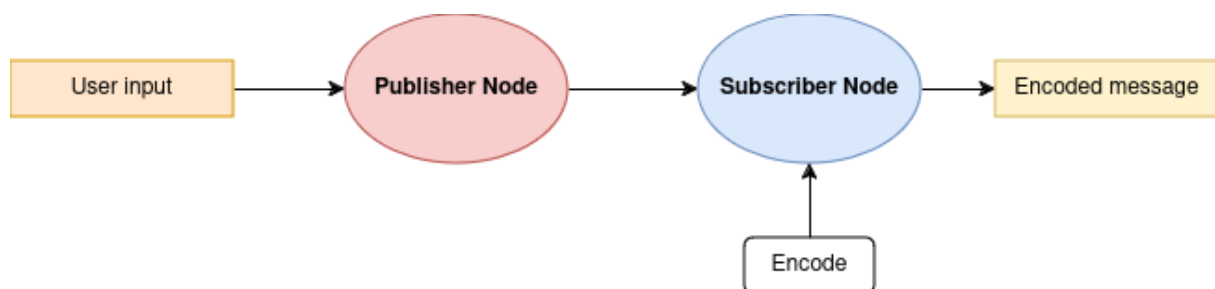


Figure 11

For asking user input and publishing it we have created a template code for you here (or clone from our github):

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class UserInputPublisher(Node):

    def __init__(self):
        super().__init__('user_input_publisher')
        self.publisher_ = self.create_publisher(String, 'user_input_topic', 10)
        self.timer = self.create_timer(1.0, self.timer_callback) # Timer to periodically call
        ↳ the callback
        self.user_input = ""

    def timer_callback(self):
        self.user_input = input("Enter a message to publish: ") # Taking input from the user
        msg = String()
        msg.data = self.user_input
        self.publisher_.publish(msg)
        self.get_logger().info(f'Publishing: "{msg.data}"')

def main(args=None):
    rclpy.init(args=args)
    node = UserInputPublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        print("Node stopped cleanly.")
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Test Phrases:

- Hello, world: **Ellohay, orldway!**
- The quick brown fox jumps over the lazy dog: **eEthay ickquay ownbray oxfay umpsjay overway ethay azylay ogday**
- Never gonna give you up: **Evernay onnagay ivegay ouyay upway**
- Never gonna let you down: **Evernay onnagay etlay ouyay ownday**
- There is an idea of a intoxication sweet spot for programming efficiency among the Mechatronics Engineers: **Erethay isway anyway ideaway ofway away intoxicationway eetsway otspay orfay ogrammingpray efficiencyway amongway ethay Echatronicsmay Engineersway.**

5.3 Fire and floods disaster land robot State Machine

The recent Bozega storm has devastated a city as it swept through the continent, causing widespread infrastructure collapse, including the failure of a chemical factory in Burger Town. Despite repeated concerns raised by the city council regarding the factory's safety hazards and inadequate disaster protocols, the factory's owners refused to implement necessary changes, opting instead to "donate" funds in response to the criticism. When the disaster struck, several units of the factory exploded, igniting chemical fires in the surrounding area. Coupled with severe flooding, the vicinity around the factory is now engulfed in flames and inaccessible due to high water levels.



Figure 12

The firefighting department reached out to the USRC seeking a solution to better assess the scene and mitigate the severity of the situation before deploying firefighters. Despite being sleep-deprived and intoxicated, the technical team proposed the idea of an autonomous land robot capable of spraying water and placing sandbags as well as mapping called Autonomous Disaster Survey Robot (ADSR) or nicknamed Lil' Gremlin. After much brainstorming, they developed a high-level design outlining the state sequence that would define the robot's behaviour.

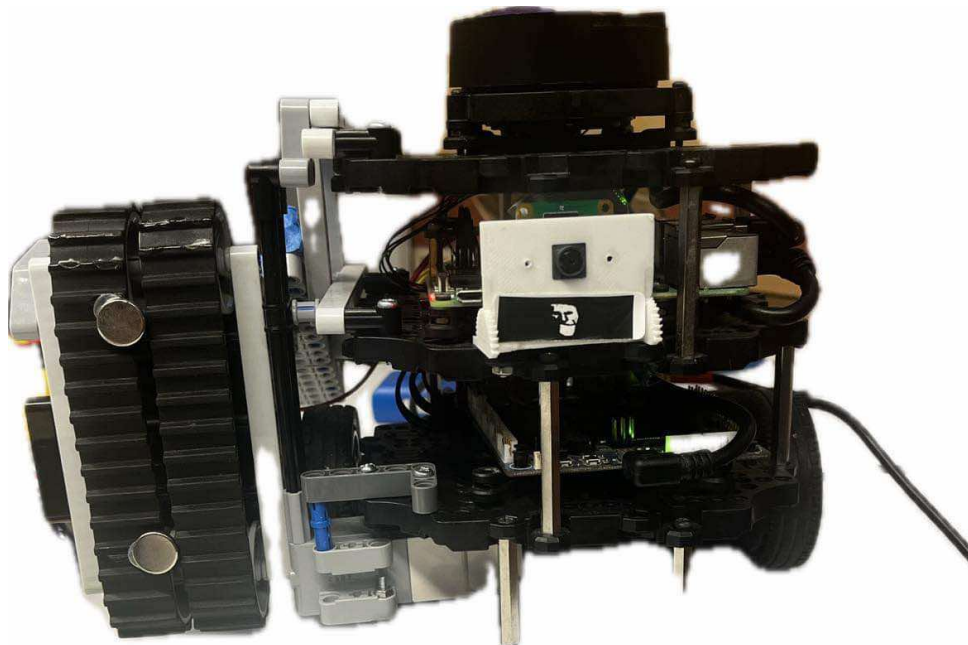


Figure 13

They have defined total of 15 states with 4 transition signals. Each of the state can be transitioned from one and another using a valid transition signal. However, for some state, the number of valid signals are different. For example, signal 4 is termed as Emergency or E, allows any previous state to transition to SOS state where the robot will stop all operations and only send out its current location to preserve the hardware components and power allowing to it to be retrieved by a team if something happens.

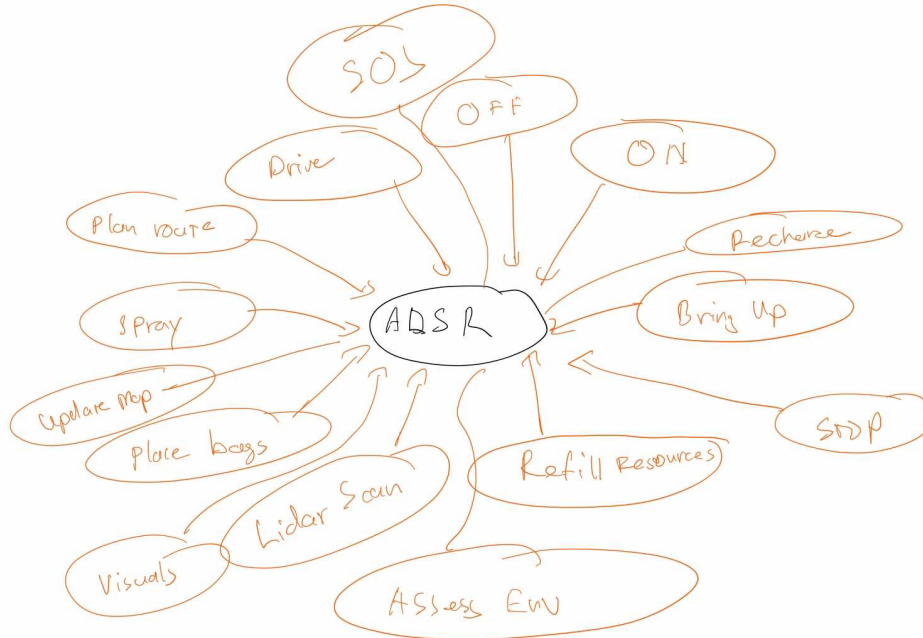


Figure 14: All states

These are all of the states and their valid transition signals

- | | |
|-------------------------------|---------------------------|
| • Power off - 1 | • Assess Environment -1,3 |
| • Power on -1 | • Update map -1,3 |
| • Bringup -1 | • Plan route -1 |
| • Recharge Battery -1,2 | • Drive -1 |
| • Refill Resources -1,2 | • Spray water -1 |
| • Stop -1,3,4 | • Place Sand Bag -1 |
| • Lidar Scan Environment -1,3 | • SOS 1,4 |
| • Capture visuals -1,3 | |

These are the transition signals

1. **Normal operation** - Syntax = *norm_op* - Sig 1
2. **Resource Low** - Syntax = *res_low* - Sig 2
3. **Re-evaluate environment** - Syntax = *re_eval* - Sig 3
4. **Emergency** - Syntax = *E* - Sig 4

Your Task

Your task is to use simple service and client in ROS2 on Python (of course) for state transitions. Depending on the context, using service to change the states of a robotic system may seem impractical since it is a blocking operation where it would stop all other operation and fulfill the request. However, for this exercise, you do not need to consider that and focus on how to create a ROS service application.

Requirements:

- You must create a **SEPARATE Service server node** that receives request for state transition by receiving and check the new **TRANSITION SIGNAL** and **NEW STATE** from a **Separate service client node**.
- Upon valid transition, the **Service Server Node** must publish the new state and transition signal to another separate subscriber node that stores the current state of the robot. At the same time it should send a response back to the client.
- New state and transition signal should **ONLY BE PUBLISHED** if they are valid.
- When the state storage node receives a valid state transition it should print out the state in a suitable message format.
- The **Service Client** nodes asks for user input: new state and transition signal. Then it sends its request to the server to await for response. If state transition request is invalid and got a **FALSE** response from the server, the client node must print out suitable error message.

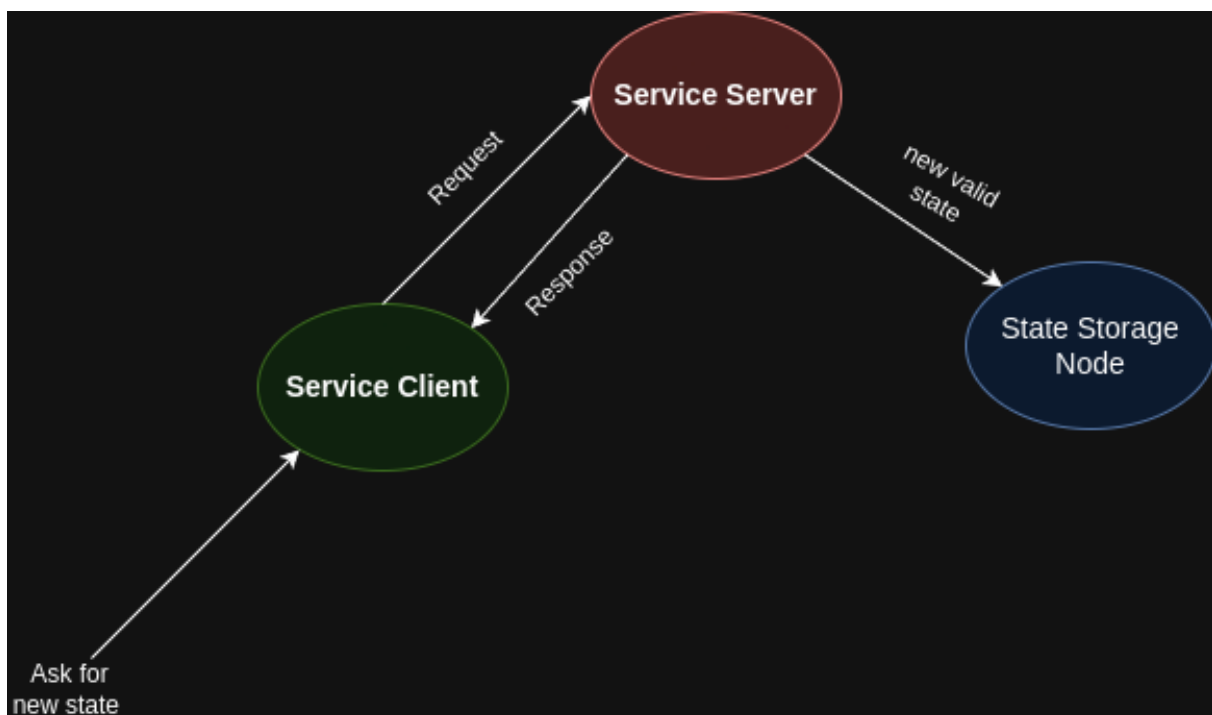


Figure 15: Required Node system design

Template code

We have provided you a detailed package code for this exercise in two ROS packages.

- *sm_srv_msg*: First package contains the custom .msg file that allows the Service Server node to publish to State Storage node once a state transition state request is valid. It also has a .srv files that defines the state transition request and boolean response upon validation.

- *state_machine*: Second package contains all scripts for service clients and servers and the state storage node. There is also a script that contains definition of the all states and their transitions. IT IS RECOMMENDED THAT YOU READ THE FILE AND UNDERSTAND HOW TO USE IT. You can modify it at your own risks.
- After cloning our github, drag the packages (folders) from Exercise folder with the aforementioned names into your src folder of your workspace like the image below.



Figure 16: Let him cook

6 Module 6: Intermediate Exercises

6.1 Maze Mapping

In this exercise, you are tasked with generating a map of a maze. You are not required to create a module for the robot to autonomously move around the maze and solve it, only traversing the entire maze and map the FULL layout of the maze. You will also have to code a node that subscribe to the topic of the robot position and plot a trajectory. You can use [trajectory_msgs](#) to figure out a way. In brief, these are the following requirements:

- Insert a the given map in Gazebo.
- Use `teleop` to move the robot around the maze and map the layout using Lidar data.
- Code a node that plots out the trajectory of the robot while it is traversing the maze.

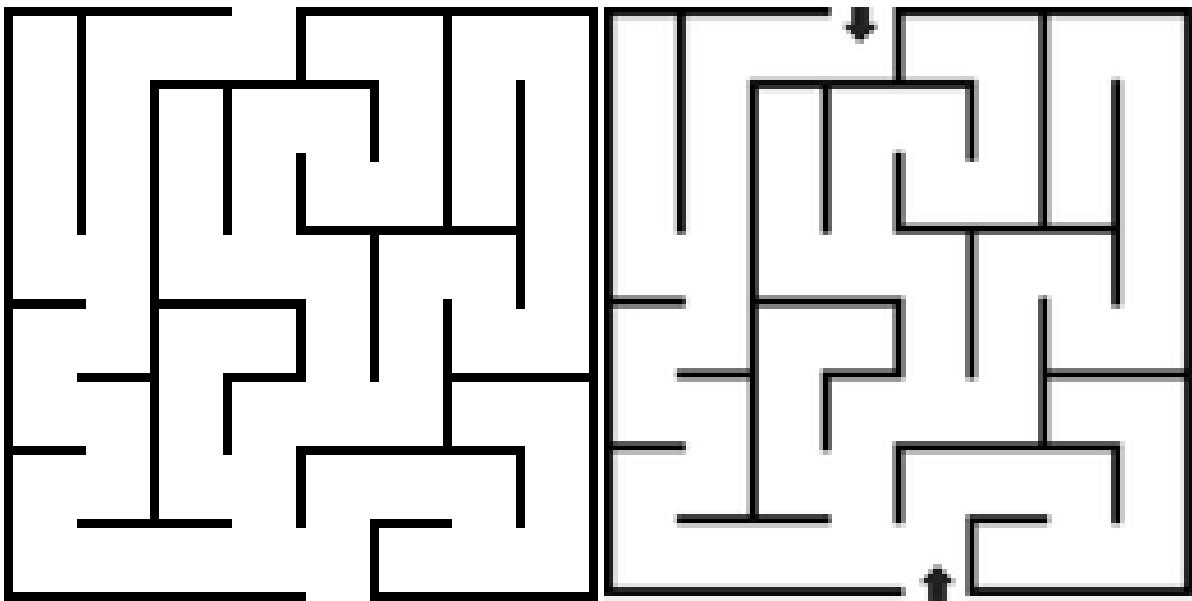


Figure 17: 8 by 8 maze

Similar to first beginner exercise in Gazebo, you will launch the following commands in different terminals

```
# Launch this first in the terminal
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_gazebo empty_world.launch.py

# Watch the instruction below on how to insert the map models in
# Open another terminal and launch mapping node
ros2 launch turtlebot3_cartographer cartographer.launch.py use_sim_time:=True

# Open another terminal to launch teleop node
ros2 run turtlebot3_teleop teleop_keyboard

# Once you have moved around the map, open another terminal to
# produce a map
ros2 run nav2_map_server map_saver_cli -f ~/map
```

You should have 4 terminal tabs opened. Now before launching Mapping node, you must insert the maze model we have made in and position it correctly. Go to the turtlebot workspace gazebo model directory by typing these commands.

```
# Go the directory where you cloned our repo
cp -r ~/ROSWorkshop/Exercises/Intermediate_Mapping
~/turtlebot3_ws/src/turtlebot3_simulations/turtlebot3_gazebo/models
```

Or you can just copy and paste it here:

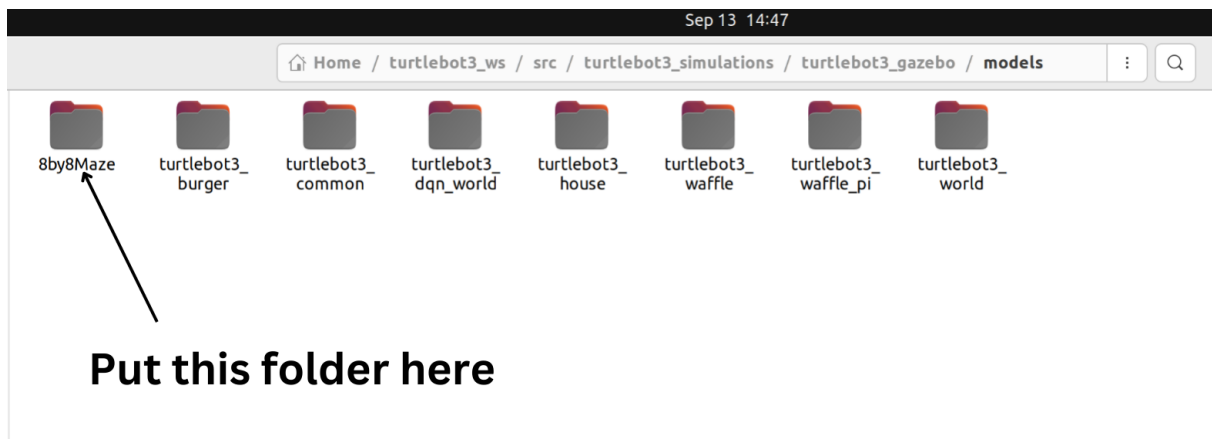


Figure 18

Once you have done that, now you should open up Gazebo by running the previous commands we gave you. **Go to Insert** and add *Maze_2*

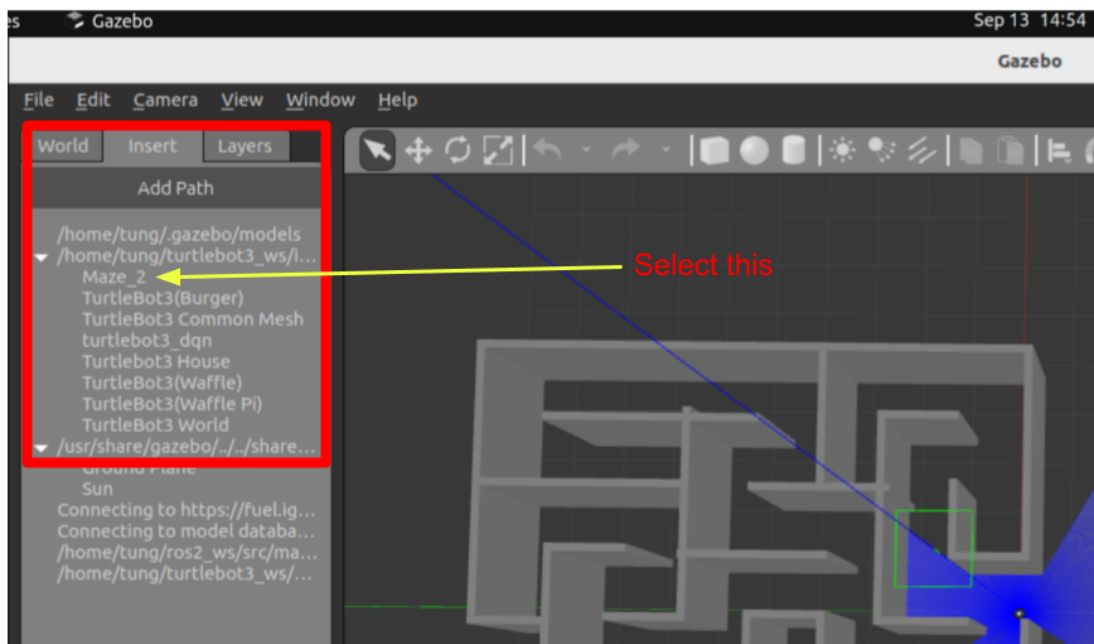


Figure 19

You can drag around the maze and place the maze so that the robot is in one of the entrance of the maze, refer to figure 17. Then you can launch the teleop node and move the robot around the maze to scan the walls to be able to produce a map of the maze.

While teleoping, you should have your trajectory node running to show where the robot's movement. We want an output that looks similar to this:

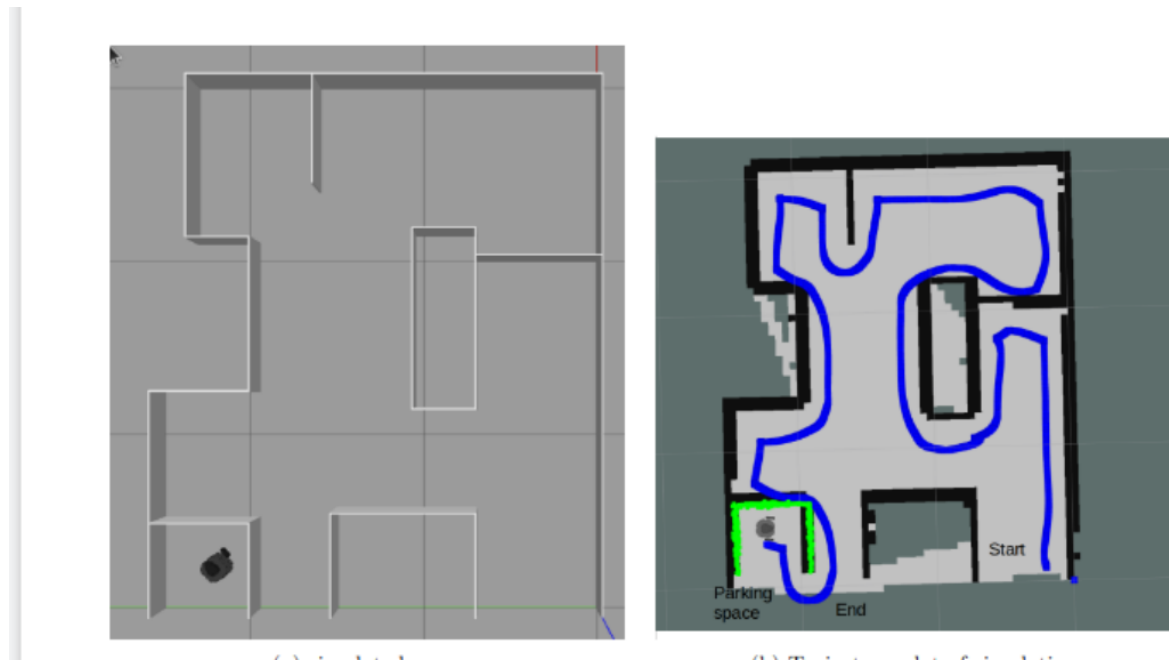


Figure 20

6.2 Perception

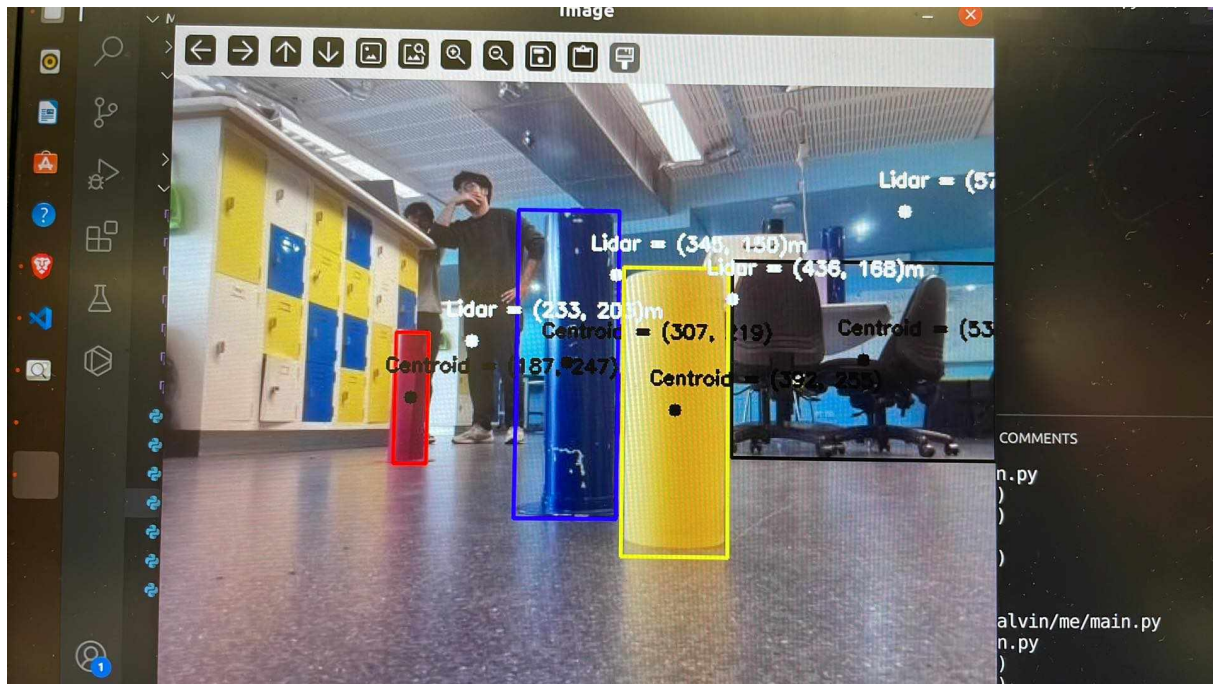


Figure 21

This exercise utilises much of the Computer Vision concepts you have learned on Day 1. While the Open CV modules were mostly theoretical, getting you to be comfortable at thresholding colours, this exercise focuses on the implementation of Computer Vision in Robotics with the extension of exploring fusing ranging data with visual data to "perceive" the environment.

Robot sensors are comprised of many different technology to allow the robots to perceive the surrounding environment. Much of the robotics perception nowadays employ the use cameras to obtain visual information combined with computer vision to find important data while depth perception can be done by either using a depth camera or combine LiDAR measurements with computer vision and associate visual data with ranging data.

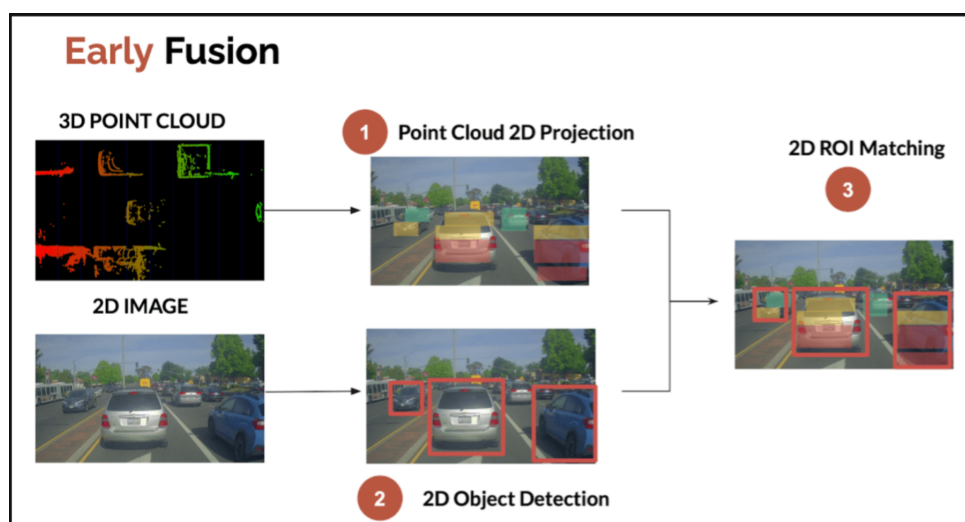


Figure 22: Computer vision and LiDAR matching

Main Goal

Our main goal is to get you to implement your computer vision modules in simulation. Using raw footage from the Turtlebot's camera, you can control the robot with your keyboard to move around and detect 8 different cylinders around the map. While doing so, whatever cylinder has been detected should be outputted in a comprehensible way in the terminal. While your camera feed should show a bounding box of the cylinders that has the same outline colour as the cylinder - see images below.

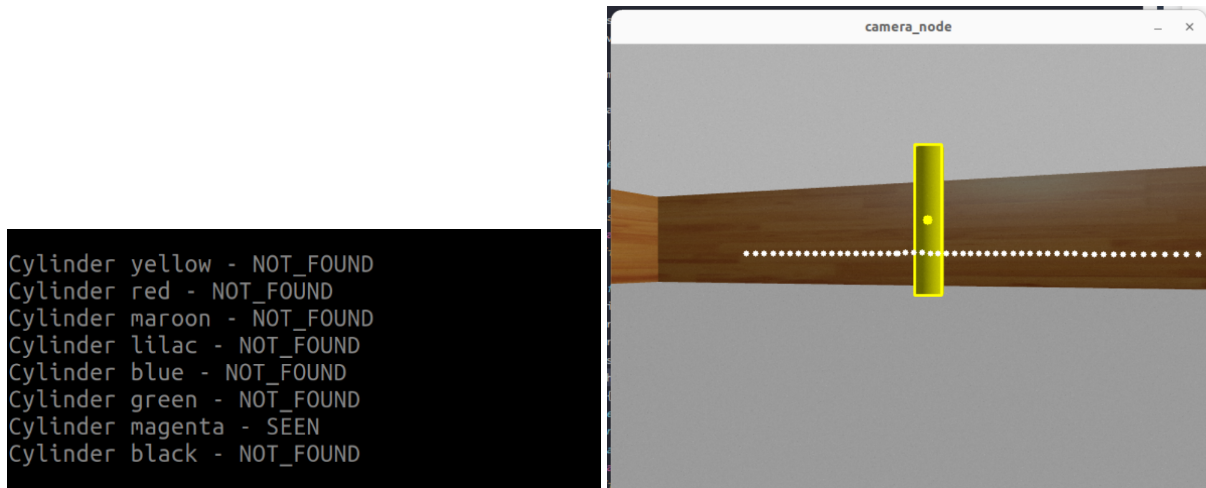


Figure 23

Later on with an extension task, you will see how LiDAR scans from the turtlebot can be used to combined with computer vision to acquire the relative range of the cylinders. In many SLAM applications, this method is widely used with landmarks to localise robot while mapping the absolute/relative positions of the surrounding features allowing the robot to autonomously plan out routes and navigate through terrains smoothly.

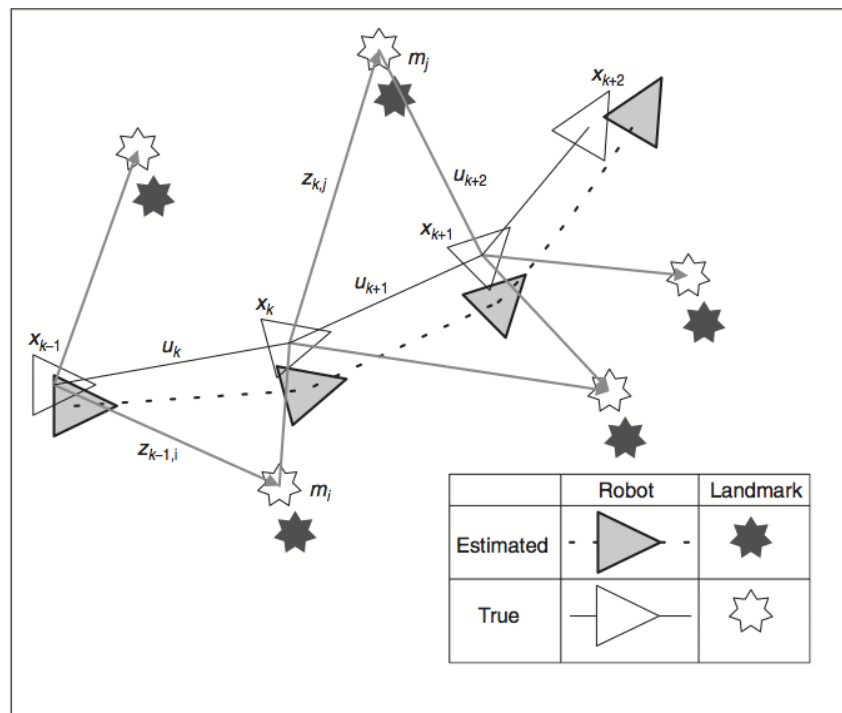


Figure 24: EKF SLAM Algorithm - Don't worry about this, just an illustration of SLAM

Camera Package Setup

We have taken codes from one of our assignments that had a perception components that consisted of both computer vision and ranging. Despite the template code has a lot of work done for you guys like defining HSV values creating an organised encapsulated data structure to store the data of the cylinders, there is still quite a lot of work to do and tune. We stripped down the codebase for you and added some TODO tag within some parts of the codes for you to complete. Read our codes carefully and understand how it works. We will give you the rundown on what each the files mean. **Once again, move this camera package to your `ros2_ws/src` folder.**

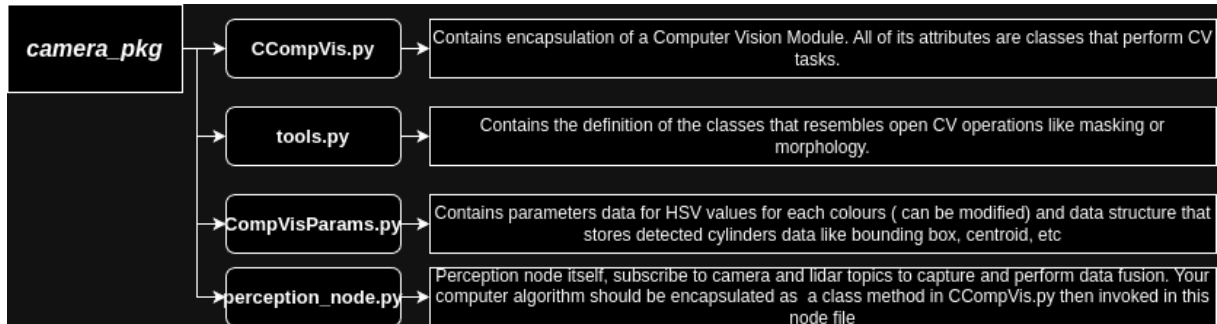


Figure 25

Gazebo World Setup

These modifications may be a bit more hectic than you think. Similar to Mapping exercise, you will need to add some new files for Gazebo to spawn up the following world:

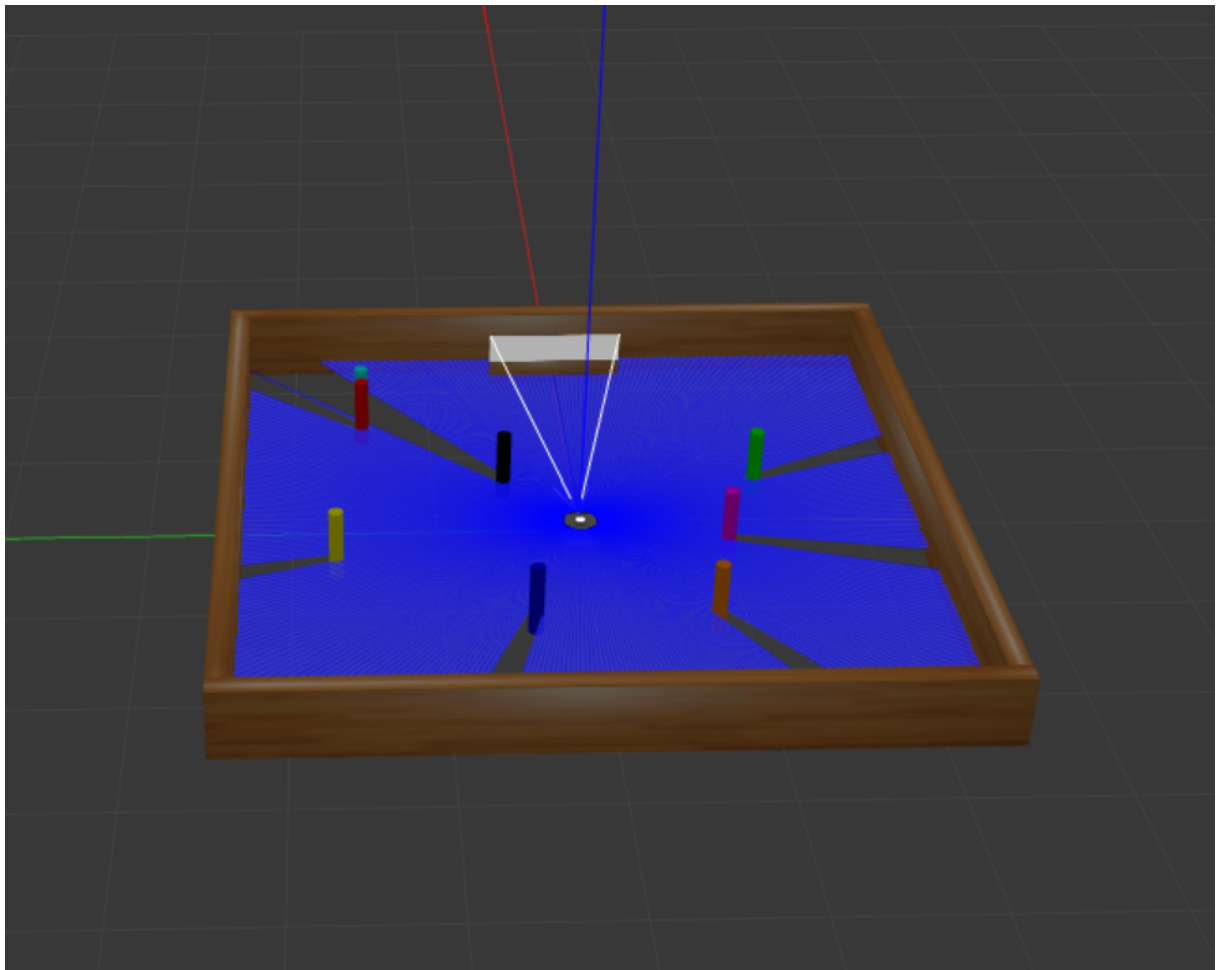


Figure 26: Gazebo world should look like this

Step 1: Navigate to this directory: `/turtlebot3_ws/src/turtlebot3_simulations/turtlebot3_gazebo`
There are 3 files you must move to the following folders. Refer to image below

- **colour_cylinders.launch.py** to launch
- **cylinders folder** to models
- **cylinders.world** to worlds

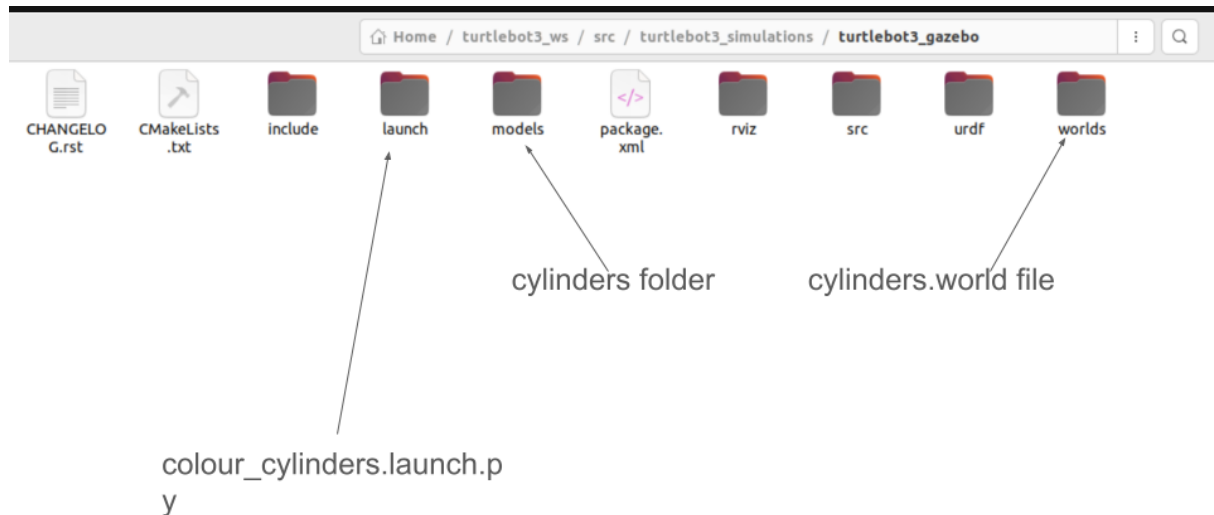


Figure 27: Where to put the files

After that is all done, enter these commands to build and launch the world.

```
cd ~/turtlebot3_ws
colcon build
source install/setup.bash

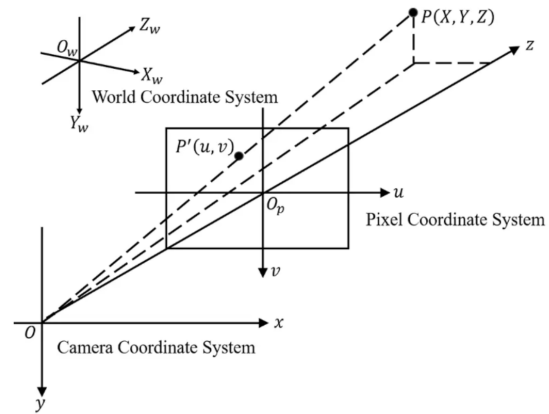
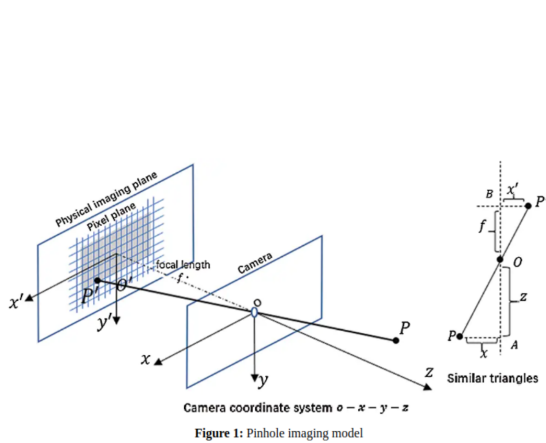
# you should be able to launch the world here
export TURTLEBOT3_MODEL=waffle_pi
ros2 launch turtlebot3_gazebo colour_cylinders.launch.py
```

Once everything is setup correctly for both the camera packages and Gazebo, you can test out the camera feed by running the nodes from the camera package we gave you.

```
cd ~/ros2_ws
colcon build
source install/setup.bash
ros2 run camera_pkg perception
```


[EXTENSION]: Sensor Fusion with LiDAR to get relative ranges of cylinders

Within the camera node code, we have provided the functionality to project lidar scans on to the camera frame. For this to happen, there is a long procedure to calibrate the camera and the LiDAR module coupled with some complex matrix transformation. However, we have done that for you so there is no need to code it from ground up. When you are running the camera node, you should see some white dots which are the projected LiDAR points. Any range that is less than a certain length is projected, you can change this to your liking. Now that you have the data and your computer vision module can detect and draw bounding boxes of the cylinders, you should be able to associate the LiDAR ranges and the detected cylinders to obtain their relative ranges. Once you have done it print the ranges out onto the terminal like how we asked you to print out detected cylinders as mentioned above.



Camera projection

$$\lambda \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} f & 0 & 0 \\ 0 & f & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

Camera **intrinsic** matrix K
(can include skew & non-square pixel size)
Camera **extrinsics**
(rotation and translation)
3D point in
world coordinates

r_2

Figure 28: Formula for projecting world points onto camera frame

To project the LiDAR points, first we must transform from word frame to camera frame then with the coordinates in camera frame we transform it again onto the image plane obtaining the pixel locations. Using these pixel locations, you can associate LiDAR ranges to the cylinders in camera frame.

Get cracking, this is quite hefty

7 Common Issues

7.1 SelectableGroups dict interface is deprecated

Issue:

```
Starting >>> module_1
--- stderr: module_1

===== warnings summary =====
test/test_flake8.py::test_flake8
test/test_flake8.py::test_flake8
  Warning: SelectableGroups dict interface is deprecated. Use select.

-- Docs: https://docs.pytest.org/en/stable/warnings.html
---
Finished <<< module_1 [0.73s]

Summary: 1 package finished [0.95s]
1 package had stderr output: module_1
```

Figure 29: Warning message example

Solution:

```
# get back to ur ros2 workspace
cd ~/ros2_ws
rosdep update
rosdep install -i --from-path src --rosdistro iron -y
colcon build # rebuild it
```

7.2 turtlebot_install.sh cannot find catkin

After running turtlebot_install.sh, student may find catkin error. Run this command to install the catkin package. [Catkin](#) is from ROS 1 but to set up the turtlebot for Gazebo, install it.

```
# run one of these three commands
sudo apt-get install python-catkin-pkg
pip install catkin_pkg
pip3 install catkin_pkg
```

7.3 AMENT_PREFIX_PATH path does not exist

```
Summary: 0 packages finished [0.08s]
[0.235s] WARNING:colcon.colcon_ros.prefix_path.ament:The path '/home/thomas/ros2_ws/install/user_input' in the environment variable AMENT_PREFIX_PATH doesn't exist
[0.235s] WARNING:colcon.colcon_ros.prefix_path.ament:The path '/home/thomas/ros2_ws/install/publisher' in the environment variable AMENT_PREFIX_PATH doesn't exist
[0.235s] WARNING:colcon.colcon_ros.prefix_path.ament:The path '/home/thomas/ros2_ws/install/listen_pkg' in the environment variable AMENT_PREFIX_PATH doesn't exist
```

Figure 30: Example of error

If this issue arises, make a .sh file and copy

```
#!/bin/bash
# Script to clean up non-existent paths from AMENT_PREFIX_PATH
```

```

# Initialize a new variable to store valid paths
NEW_AMENT_PREFIX_PATH=""

# Split the AMENT_PREFIX_PATH by colon and iterate over each path
IFS=':' read -ra PATHS <<< "$AMENT_PREFIX_PATH"
for PATH in "${PATHS[@]}"; do
    # Check if the path exists and is a directory
    if [ -d "$PATH" ]; then
        # Append valid paths to the new AMENT_PREFIX_PATH variable
        NEW_AMENT_PREFIX_PATH="${NEW_AMENT_PREFIX_PATH}:$PATH"
    else
        # Print a message if the path does not exist
        echo "Removing non-existent path: $PATH"
    fi
done

# Remove the leading colon if NEW_AMENT_PREFIX_PATH is not empty
NEW_AMENT_PREFIX_PATH=${NEW_AMENT_PREFIX_PATH#.}

# Update the AMENT_PREFIX_PATH with the cleaned-up version
export AMENT_PREFIX_PATH="$NEW_AMENT_PREFIX_PATH"

# Print the updated AMENT_PREFIX_PATH
echo "Updated AMENT_PREFIX_PATH: $AMENT_PREFIX_PATH"

```